CEH v13 AI (312-50) Study Guide - Module 11: Session Hijacking

EC-Council CEH v13 AI (312-50) Exam Prep

Study guide built from official module screenshots

Table of Contents

1. Module at a Glance
2. Big Picture Map
3. Topic-by-Topic Guide
   - 3.1 Module Title and Learning Objectives [page 1]
   - 3.2 What is Session Hijacking? [page 2]
   - 3.3 Session Hijacking Process [page 2]
   - 3.4 Session Hijacking in the OSI Model [page 3]
   - 3.5 Spoofing vs Hijacking [page 3]
   - 3.6 Session Basics: What a Session ID Is and How It Gets Stolen (ADDED CONTEXT) [slides 3, 8, 9 not in screenshots]
   - 3.7 How to Predict a Session Token [page 4]
   - 3.8 Compromising Session IDs Using Man-in-the-Middle / Manipulator-in-the-Middle Attack [page 4]
   - 3.9 Compromising Session IDs Using Man-in-the-Browser / Manipulator-in-the-Browser Attack [page 5]
   - 3.10 Compromising Session IDs Using Session Replay Attacks [page 5]
   - 3.11 Compromising Session IDs Using Session Fixation [page 6]
   - 3.12 Session Hijacking Using Proxy Servers [page 6]
   - 3.13 Session Hijacking Using the CRIME Attack (ADDED CONTEXT) [slide 16 not in screenshots]
   - 3.14 Session Hijacking Using the Forbidden Attack [page 7]
   - 3.15 Session Hijacking Using Session Donation Attack [page 7]
   - 3.16 Network-Level Session Hijacking [page 8]
   - 3.17 TCP/IP Hijacking [page 8]
   - 3.18 IP Spoofing: Source Routed Packets [page 9]
   - 3.19 RST Hijacking [page 9]
   - 3.20 Blind Hijacking [page 10]
   - 3.21 UDP Hijacking [page 10]
   - 3.22 MiTM Attack Using Forged ICMP and ARP Spoofing [page 10]
   - 3.23 PetitPotam Hijacking [page 11]
   - 3.24 Session Hijacking Tools [page 11]
   - 3.25 Protecting against Session Hijacking [page 12]
   - 3.26 Session Hijacking Detection Tools [page 12]
   - 3.27 Approaches to Prevent Session Hijacking: HSTS and Token Binding [page 13]
   - 3.28 Approaches to Prevent MITM Attacks: DNS over HTTPS [page 13]

3.29 IPsec [page 14]
3.30 IPsec (Cont'd): Modes and Architecture [page 14]
3.31 Cookie Flags and Server-Side Session Hygiene (ADDED CONTEXT) [no slide]
3.32 AI and Session Hijacking (ADDED CONTEXT) [no slide]
4. Tool Encyclopedia
5. Command Sheet (I have no lab, so teach me to read commands)
5.1 tcpdump (named on the RST hijacking slide)
5.2 hping3 (ADDED CONTEXT, packet crafting for hijack labs)
5.3 Ettercap / arpspoof (ADDED CONTEXT, for the ARP spoofing MITM slide)
5.4 Wireshark session-hijacking display filters (ADDED CONTEXT)
5.5 SSH / OpenSSH and the Strict-Transport-Security header (ADDED CONTEXT)
6. Ports, Protocols, Standards and Acronyms
6.1 Ports and protocols seen in the screenshots and command sheet
6.2 Standards and acronyms
7. Attack vs Countermeasure and Look-alike Comparisons
7.1 Attack and its countermeasure
7.2 Look-alike pairs
8. Exam Cheat Sheet
8.1 Keyword to answer
8.2 Must-memorize lists
8.3 Numbers and defaults
8.4 Rules for picking the BEST option
9. Practice Questions
10. Flashcards
11. Glossary
12. Coverage Report
12.1 Map of pages to guide sections
12.2 Items marked [UNCLEAR]
12.3 Items to double check (gaps and verify markers)
12.4 Coverage confirmation

CEH v13 AI (312-50) Study Guide - Module 11: Session Hijacking

Source: 14 pages of screenshots from the official Module 11 theory videos (slide numbers 1 to 34 in the deck, several repeated across pages). Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small or blurry to read with full confidence.

1. Module at a Glance

Module: 11 - Session Hijacking (EC-Council CEH v13 AI).

What this module is (5 lines):

1. Session hijacking means an attacker seizes control of a valid, already-authenticated TCP session between two computers, instead of logging in with stolen credentials of their own.
2. The module splits hijacking into two levels: application-level (stealing or guessing the HTTP session ID) and network-level (attacking the TCP/IP or UDP connection itself).
3. Application-level methods covered: predicting a session token, MITM, Man-in-the-Browser, session replay, session fixation, proxy server hijacking, the Forbidden attack, and session donation.
4. Network-level methods covered: TCP/IP hijacking, IP spoofing with source-routed packets, RST hijacking, blind hijacking, UDP hijacking, MITM using forged ICMP and ARP spoofing, and the modern PetitPotam attack against Active Directory.
5. The module ends with tools, a 14-point defense checklist, detection tools, HSTS, Token Binding, DNS over HTTPS, and IPsec.

Why it matters: Session hijacking sits in the "gaining access" and "maintaining access" phases of the five hacking phases. Most web logins only check a username and password once, at the start of a session; after that, a session ID (a cookie, a URL token, or a hidden form value) is the only proof of identity. If an attacker can steal, guess, or trick a victim into using an attacker-known session ID, they skip the password entirely. The exam leans on this module for scenario questions because many named attacks produce the same visible symptom (an unauthorized user acting as someone else), so you must read the mechanism in the question carefully.

What the exam usually asks from it:

- Spoofing vs hijacking: did the attacker start a new session with stolen credentials (spoofing), or take over an existing one (hijacking)?
- Which attack matches a description: MITM, Man-in-the-Browser, session replay, session fixation, proxy server hijacking, Forbidden attack, session donation.

- Application-level vs network-level hijacking, and which OSI/TCP-IP layer each one works at.
- Network-level attack recall: TCP/IP hijacking, source-routed IP spoofing, RST hijacking, blind hijacking, UDP hijacking, PetitPotam.
- Reading a TCP sequence/acknowledgment number diagram to spot where an attacker injects data.
- Tool recall: Hetty, Caido, bettercap, OWASP ZAP, WebSploit Framework, sslstrip, Burp Suite, Wireshark, Colasoft Packet Builder, tcpdump.
- Countermeasure matching: SSH, session ID rules, timeouts, HSTS, Token Binding, DNS over HTTPS, IPsec (AH vs ESP, transport vs tunnel mode).

The 10 things to remember most:

1. Session hijacking = taking over an existing, already-authenticated TCP session. Spoofing = starting a brand-new session using stolen credentials. Hijacking relies on a legitimate user doing the login.
2. The five-step hijacking process, bottom to top: Sniff the network, Monitor packets and predict the sequence number, Desynchronize the victim's session, predict the Session ID to take over the session, then inject Commands into the target server.
3. Network-level hijacking attacks transport/internet protocols (TCP, UDP, IP). Application-level hijacking attacks the HTTP session ID directly. Network-level info often feeds an application-level attack.
4. A session token can be predicted if it is sequential, timestamp-based, drawn from a small token space (brute-forceable), or generated by a weak/predictable pseudo-random number generator (PRNG), especially with no rate limiting on guesses.
5. Application-level techniques: MITM (splits one TCP connection into two), Man-in-the-Browser (a Trojan-installed malicious browser extension), Session Replay (captures and resends an auth token), Session Fixation (victim is tricked into using an attacker-known SID), Proxy Server hijacking (attacker relays traffic as a transparent proxy), Forbidden attack (TLS nonce reuse), Session Donation (attacker donates their own logged-in SID to the victim).
6. Network-level techniques: Blind hijacking (inject with no visibility of the response), UDP hijacking (race to answer first), TCP/IP hijacking (spoofed packets plus sequence number prediction), RST hijacking (forged reset packet), Man-in-the-middle with a packet sniffer, and IP spoofing with source-routed packets.
7. PetitPotam forces a domain controller to authenticate to an attacker's server over MS-EFSRPC, then relays that NTLM authentication to Active Directory Certificate Services (AD CS) to mint a certificate with admin-level privileges.
8. The 14-point session hijacking countermeasure list centers on: unique server-generated session IDs, encryption everywhere (SSH, VPN, IPsec), logout and timeout functions, never putting the session ID in a URL, and switching hubs for switches.

9. HSTS forces a browser to use HTTPS only. Token Binding cryptographically ties a session token to the specific TLS connection it was issued on, so a stolen token is useless on another connection.
10. IPsec secures IP packets with two headers: AH (Authentication Header, authentication only) and ESP (Encapsulating Security Payload, authentication plus encryption). It runs in Transport mode (host to host) or Tunnel mode (gateway to gateway, whole packet wrapped).

AI angle (module-wide): The screenshots do not show a dedicated AI slide for this module, unlike modules 2 to 5. ADDED CONTEXT: in CEH v13 AI, attackers can still use AI assistants to speed up two parts of this module: spotting predictable patterns in a batch of captured session tokens (feeding samples to an AI tool and asking it to find the pattern), and generating realistic phishing lures for session fixation or session donation attacks (the email text luring a victim to click a crafted link). Defenders can use AI-driven log analysis (the detection tools in section 3.23 already use machine learning) to flag abnormal sequence numbers, duplicate session IDs from two different IP addresses, or sudden spikes in token-guessing traffic.

2. Big Picture Map

```
SESSION HIJACKING (Module 11)
|
+-- CONCEPTS
|    What it is -> 5-step process -> OSI view
|    (network-level vs application-level)
|    -> Spoofing vs Hijacking
|
+-- APPLICATION-LEVEL (steal/guess the session ID)
|    Predict token -> MITM -> Man-in-the-Browser
|    -> Session Replay -> Session Fixation
|    -> Proxy Server -> Forbidden Attack
|    -> Session Donation
|
+-- NETWORK-LEVEL (attack TCP/UDP/IP itself)
|    TCP/IP hijacking -> IP spoofing (source routed)
|    -> RST hijacking -> Blind / UDP hijacking
|    -> MITM via forged ICMP/ARP -> PetitPotam
|
+-- TOOLS, DEFENSE AND DETECTION
     Hetty, Caido, Burp Suite, bettercap...
     -> 14-point defense checklist
     -> Detection tools (Wireshark, USM, IPS)
     -> HSTS, Token Binding, DoH, IPsec
```

How the two levels compare at a glance:

Level	Target	Needs victim to log in first?
Application-level	The HTTP session ID (cookie, URL, form)	Usually yes (fixation, donation)
Network-level	TCP, UDP, or IP itself	Victim already mid-session

Read the map top to bottom: first learn what hijacking is and how it differs from spoofing, then the eight ways to attack a session ID directly at the application layer, then the six ways to attack the underlying network connection, and finally the tools, the big defense checklist, and the specific modern protections (HSTS, Token Binding, DNS over HTTPS, IPsec).

3. Topic-by-Topic Guide

The topics below follow the order of the screenshots.

3.1 Module Title and Learning Objectives [page 1]

What it is (simple words): The module is "Module 11 - Session Hijacking". Four learning objectives: summarize session hijacking concepts, explain application-level session hijacking, explain network-level session hijacking, and explain session hijacking countermeasures.

Analogy: A relay race. Concepts explain what a "baton pass" (session) is and how someone could steal the baton mid-race. Application-level and network-level are the two different places on the track where the baton can be grabbed. Countermeasures are the rules that stop a stranger from grabbing it.

How the module fits the hacking flow (ADDED CONTEXT):

1. Modules 2 to 5 gather information from outside the network.
2. Module 6 (System Hacking) and Module 7 (Malware Threats) get an initial foothold, and Module 8 (Sniffing) captures LAN traffic.
3. Module 11 (Session Hijacking) uses captured traffic, a position on the network, or a tricked victim to take over an already-authenticated session, which usually means gaining access without ever knowing the victim's password.
4. Later modules build on this: Module 13/14 (Web Servers/Applications) reuse these same session ID attacks against web apps specifically, and Module 16 (Wireless) repeats MITM-style hijacking over Wi-Fi.

Real-world use: A penetration tester checks whether a client's internal banking portal can be hijacked by sniffing an employee's HTTP session on the office LAN, since the four objectives map directly onto the test plan: understand the concept, try application-level attacks, try network-level attacks, then write up countermeasures.

Tools and commands used here: None. This slide is the plan.

EXAM FOCUS:

- The exam tests each of the four objective blocks separately across many questions, not as a single list question.

- Trap: do not confuse this module's "hijacking" with "spoofing" from Module 8/ Module 11 itself; section 3.5 below exists specifically to settle that confusion.

Memory trick: "C-A-N-C": Concepts, Application-level, Network-level, Countermeasures.

3.2 What is Session Hijacking? [page 2]

What it is (simple words): Session hijacking is an attack in which an attacker seizes control of a valid TCP communication session between two computers. Because most authentications only occur at the start of a TCP session, this lets the attacker gain access to a machine without ever authenticating themselves the normal way. Attackers can sniff all the traffic from an established TCP session and perform identity theft, information theft, and fraud. The attacker steals a valid session ID and uses it to authenticate as if they were the real user.

Analogy: A conference badge that gets you into every room for the day. The guard only checks your ID once at the front desk; after that, anyone holding your badge can walk into any room as "you". Session hijacking is stealing that badge after the front-desk check already happened.

How it works (numbered steps, from the diagram):

1. The victim sends credentials to the web server (credential transmission).
2. The server confirms the credentials (credential confirmation) and sets up the session (session setup).
3. The victim and server exchange normal traffic: data requests and data transmissions, back and forth, for the life of the session.
4. The attacker sniffs this traffic.
5. Using the sniffed session information, the attacker sends their own data request to the server and receives a data transmission back, as if they were the victim.

Real-world use: An attacker on a coffee shop Wi-Fi network sniffs an unencrypted HTTP session of a customer logged into a webmail account. Because the login already happened, the attacker's sniffed session cookie is enough to read the victim's inbox without ever seeing a password.

Tools and commands used here: Named later in sections 4 and 5 (packet sniffers, MITM proxies).

Countermeasures and defense: See section 3.22 for the full 14-point list; in short, encrypt sessions end to end and never trust a session ID alone for the life of a long session.

EXAM FOCUS:

- Keyword "seizes control of a valid/existing TCP session" -> session hijacking, not spoofing (see section 3.5).
- Keyword "authentication only happens once, at the start" -> this is exactly why hijacking works; it is a favorite exam fact.
- Trap: a question may describe credential theft via phishing and call it "hijacking" loosely; check whether the attacker reused an existing session or started a brand-new one with stolen credentials.

Memory trick: "Hijacking steals the badge after the guard already checked it."

3.3 Session Hijacking Process [page 2]

What it is (simple words): A five-stage process that most session hijacking attacks follow, shown bottom to top on the slide.

How it works (numbered steps):

1. **Sniff:** Place yourself between the victim and the target; you must be able to sniff the network.
2. **Monitor:** Monitor the flow of packets and predict the sequence number.
3. **Session Desynchronization:** Break the connection to the victim's machine.
4. **Session ID Prediction:** Take over the session.
5. **Command Injection:** Start injecting packets into the target server.

Analogy: Hijacking a moving train. First you walk alongside the tracks and watch (sniff). Then you time the train's schedule exactly (monitor and predict sequence numbers). Then you pull the original driver off at a signal stop (desynchronize). Then you climb into the driver's seat using the schedule you memorized (predict the session ID). Finally you start driving the train wherever you want (inject commands).

Real-world use: An attacker on a shared office switch first ARP- poisons the switch to see a victim's traffic (sniff), watches the TCP sequence numbers of an open Telnet session (monitor), sends a reset to drop the victim's connection (desynchronize), guesses the next sequence number to slide into the conversation (session ID prediction), and then types commands as if they were the victim (command injection).

Tools and commands used here: Packet sniffers (Wireshark, tcpdump), packet crafting tools (Colasoft Packet Builder, hping3), all detailed in sections 4 and 5.

Countermeasures and defense: Breaking any one stage stops the whole chain: switched networks and encryption make sniffing harder; strong, server-only session IDs make prediction harder; timeouts and logout limit the window of opportunity.

EXAM FOCUS:

- Must-memorize the five stages in order: Sniff, Monitor, Session Desynchronization, Session ID Prediction, Command Injection.
- Trap: "desynchronization" is sometimes confused with "denial of service"; here it specifically means breaking the victim's side of the TCP connection so the attacker's spoofed packets are accepted instead.
- Keyword "predict the sequence number" -> this stage is Monitor, the stage right before desynchronization.

Memory trick: "SMS-SC": Sniff, Monitor, Session-desync, Session-ID prediction, Command injection. Read it as "a text message (SMS) starts the Session Commandeering (SC)".

3.4 Session Hijacking in the OSI Model [page 3]

What it is (simple words): The module splits session hijacking into two levels based on where in the OSI model the attack happens.

- **Network-Level Hijacking:** the interception of packets during transmission between a client and a server in a TCP or UDP session.
- **Application-Level Hijacking:** gaining control over the HTTP user session by obtaining the session IDs.

Analogy: A delivery truck (network-level) carries a sealed package (application-level). Hijacking the truck on the highway is network-level hijacking; the cargo (TCP/UDP) is what is being intercepted. Stealing the tracking number off the package itself, so you can claim it at the delivery counter, is application-level hijacking; the HTTP session ID is the "tracking number".

How it works (numbered steps):

1. Decide what is being attacked: the transport/internet protocol carrying the session (network-level), or the HTTP-layer session ID itself (application-level).
2. Network-level attacks (section 3.14 onward) need the attacker to manipulate TCP sequence numbers, UDP timing, or IP routing.
3. Application-level attacks (sections 3.6 to 3.13) need the attacker to steal, predict, fix, replay, or donate a session ID value.
4. The two levels can combine: network-level sniffing often gathers the exact information (sequence numbers, session cookies in cleartext) that an application-level attack then uses.

Real-world use: A red team first runs an ARP spoofing MITM attack (network-level, see section 3.19) purely to sniff traffic, then uses the captured HTTP cookie from that traffic to directly log in to a web app (application-level session replay, section 3.9).

Tools and commands used here: Network-level: packet sniffers, packet crafting tools. Application-level: browser proxies, MITM HTTP tools (Burp Suite, OWASP ZAP).

EXAM FOCUS:

- Keyword "HTTP session ID", "cookie", "web session" -> Application-level hijacking.
- Keyword "TCP session", "UDP session", "packet interception during transmission" -> Network-level hijacking.
- Trap: a question may describe an attack that sounds network-level (packet sniffing) but the actual goal stated is stealing an HTTP cookie; classify by the goal of the attack, not just the mechanism used to get there.

Memory trick: "Network level = the pipe. Application level = the password riding inside the pipe."

3.5 Spoofing vs Hijacking [page 3]

What it is (simple words): Two attacks that can look similar but work differently.

- **Spoofing Attack:** An attacker pretends to be another user or machine (the victim) to gain access. The attacker does not seize control of an existing active session; instead, they initiate a new session using the victim's stolen credentials.
- **Hijacking:** The process of seizing control of an existing active session. The attacker relies on the legitimate user to create a connection and authenticate first.

Analogy: Spoofing is forging someone's ID card and walking up to the front desk yourself to get a brand-new visitor badge in their name. Hijacking is waiting until the real person has already walked through security with their own badge, then taking the badge away from them mid-visit.

How it works (numbered steps, from the diagram):

1. *Spoofing:* James (victim) has credentials stolen by John (attacker).
2. John starts a brand-new session, presenting James' stolen credentials directly to the server ("I am James, here are my credentials").
3. *Hijacking:* James logs on to the server with his own credentials, and a real session is established.
4. John predicts the TCP sequence number and kills James' connection.
5. John spoofs James' IP address and hijacks the already-active session.

Real-world use: A phishing attack that tricks a user into typing their password into a fake login page is spoofing (a new session, stolen credentials). An attacker who ARP-poisons the same user's already-open banking session and takes it over mid-transaction is hijacking (an existing session, no password needed).

Tools and commands used here: Spoofing commonly pairs with credential-harvesting phishing kits; hijacking pairs with sniffers and packet crafting tools (sections 4 and 5).

Countermeasures and defense: Anti-phishing training and multi-factor authentication reduce spoofing. Session ID hardening, encryption, and timeouts (section 3.22) reduce hijacking.

EXAM FOCUS:

- This is one of the most common look-alike pairs on the exam; see section 7.2 for the dedicated comparison entry.
- Keyword "initiates a new connection using stolen credentials" -> Spoofing.
- Keyword "takes over an existing, already-authenticated connection" -> Hijacking.
- Trap: both attacks can end with the attacker "being" the victim to the server; the exam question differentiates them by whether a NEW session was created or an EXISTING one was seized.

Memory trick: "Spoof = fake ID at the front desk. Hijack = mug the visitor after they already got their badge."

3.6 Session Basics: What a Session ID Is and How It Gets Stolen (ADDED CONTEXT) [slides 3, 8, 9 not in screenshots]

> **ADDED CONTEXT:** Your screenshots skip slide numbers 3, 8 and 9. They fall between the objectives, the OSI slide, and "How to Predict a Session Token". They are probably divider or overview slides. This section gives the background every later topic needs. It is not from your notes.

What it is (simple words):

- **HTTP** (Hypertext Transfer Protocol) is stateless. The server forgets you after every request.
- A **session** is the server's way of remembering you after login.
- A **session ID** (SID, also called a session token) is a random-looking value the server gives you at login. Your browser sends it back with every request. The server uses it to know "this is the same user".
- A SID travels in one of three places: a **cookie**, the **URL** (query string), or a **hidden form field**. These are the same three places the session fixation slide lists (section 3.11).
- **Cookie:** a small piece of data the server asks the browser to store and send back on later requests.

Analogy: A coat check ticket. You hand over your coat once (login). You get a numbered ticket (session ID). Later, anyone holding the ticket gets the coat. The attendant never checks your face again.

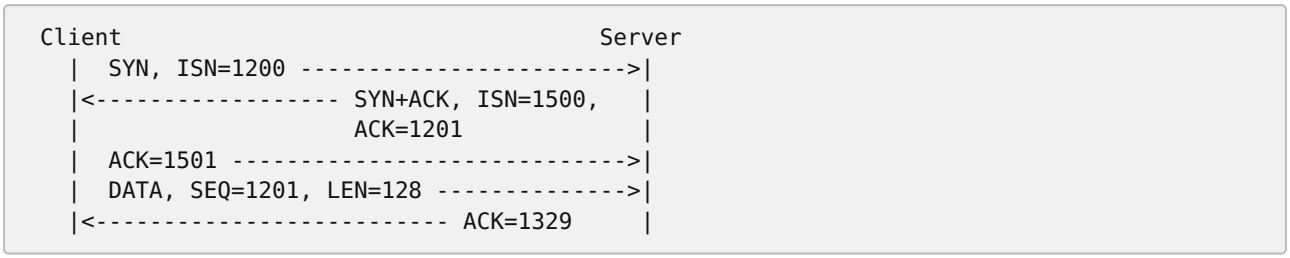
How a session works (numbered steps):

1. You send your username and password over HTTP or HTTPS.
2. The server checks them and creates a session record.
3. The server sends back a SID, usually in a `Set-Cookie` header.
4. Your browser stores the cookie and sends it on every later request.
5. The server looks up the SID and treats you as the logged-in user.
6. The session ends at logout or after a timeout, and the server should delete the record.

TCP sequence numbers (needed for network-level attacks):

TCP (Transmission Control Protocol) is the reliable transport protocol under HTTP. It numbers every byte so both sides can detect loss and reordering.

- **ISN** (Initial Sequence Number): the random starting number each side picks during the three-way handshake.
- **SEQ** (sequence number): the number of the first data byte in this packet.
- **ACK** (acknowledgment number): the next byte the receiver expects. It equals the last SEQ plus the data length.
- **Window (WIN):** how many bytes the receiver will accept before it must send an ACK.



The slide in section 3.17 uses exactly these numbers. The check is simple: 1201 + 128 = 1329.

Ways an attacker gets a session ID (overview):

Method	One-line idea	Where in this guide
Sniffing	Read the SID from unencrypted traffic	3.2, 3.8
Prediction	Guess the next SID from a pattern	3.7
Brute force	Try every possible SID in a small space	3.7
MITM	Sit in the middle and copy the SID	3.8
Malware / MitB	Trojan reads or edits the browser	3.9

Method	One-line idea	Where in this guide
Replay	Resend a captured token	3.10
Fixation	Make the victim use a known SID	3.11
Donation	Make the victim use the attacker's SID	3.15
XSS (ADDED)	Script reads `document.cookie`	3.31

Active vs passive hijacking (ADDED CONTEXT):

- **Passive hijacking:** the attacker takes over the session but only watches (records) the traffic. The victim is not cut off.
- **Active hijacking:** the attacker finds an active session and takes it over. The victim is usually knocked offline (desynchronized, section 3.3) so only the attacker's packets are accepted.

Why session hijacking succeeds (ADDED CONTEXT, verify exact EC-Council wording):

1. No account lockout for invalid session IDs, so an attacker can guess forever.
2. Weak session ID generation or very short IDs.
3. Indefinite session expiration (the SID never dies).
4. Session IDs sent in cleartext (no encryption).
5. Insecure handling of SIDs, such as putting them in URLs.
6. TCP/IP itself has no built-in authentication of packets, and most countermeasures fail unless traffic is encrypted.

Real-world use: A tester reviews a client's web app. They log in, copy the SID cookie, log out, and replay the same cookie. The server still accepts it. That is finding number 3 (indefinite expiration) and 5 (logout does not destroy the session).

Tools and commands used here: Browser developer tools (F12), Burp Suite, OWASP ZAP (section 4). Reading cookies in browser developer tools is passive and safe on your own accounts.

AI angle: An AI assistant can explain a captured `Set-Cookie` header line by line. Example prompt: "Explain each attribute in this header and tell me which security flags are missing: Set-Cookie: JSESSIONID=ab12; Path=/".

Countermeasures and defense: Generate SIDs on the server only, make them long and random, regenerate after login, expire them, and send them only over HTTPS (full list in section 3.25).

EXAM FOCUS:

- Keyword "stateless protocol" -> HTTP; the SID is what adds state.

- ACK = previous SEQ + data length. Expect one arithmetic question.
- Trap: a SID in a cookie and a SID in a URL are both session IDs. The URL version is more dangerous because it leaks in logs, bookmarks, and the Referer header (ADDED CONTEXT).

Memory trick: "Cookie, URL, Hidden" - the three hiding spots of a SID (CUH, say "cuh, there it is").

3.7 How to Predict a Session Token [page 4]

What it is (simple words): A session token is only safe if nobody can guess it. If tokens follow a pattern, an attacker can work out other users' tokens. The slide lists four weaknesses that make tokens predictable.

1. **Sequential tokens:** the token counts up. The slide example: `text http://www.certifiedhacker.com/view/JBEX1001 http://www.certifiedhacker.com/view/JBEX1002 http://www.certifiedhacker.com/view/JBEX1003` `JBEX` is the constant part. `1001`, `1002`, `1003` is the sequential part. After seeing 1003 the attacker simply tries 1004.
2. **Timestamp-based tokens:** the token contains the date and time. The slide example: `text http://www.certifiedhacker.com/view/JBEX20240611T1234 http://www.certifiedhacker.com/view/JBEX20240611T1236 http://www.certifiedhacker.com/view/JBEX20240611T1238` `JBEX` is constant, `20240611` is the date (11 June 2024), and `T1234` is the time. Times rise by two, so the next one is `T1240`.
3. **Brute force on a small token space:** if the token has only a few possible values, the attacker tries them all. The slide example shows short numeric tokens (`0011`, `0033`, `0055`) marked "small token size". A 4-digit number has only 10,000 possibilities.
4. **Weak random number generators:** a **PRNG** (pseudo-random number generator) makes numbers from a formula and a starting value called a **seed**. If the PRNG is predictable, and the attacker learns the seed or the algorithm, they can compute every future token.

The slide also adds one more cause: **lack of rate limiting.** Rate limiting means the server blocks or slows a client that makes too many requests. Without it, an attacker can make thousands of token guesses per minute without being blocked.

Analogy: A hotel that numbers room keys 101, 102, 103 in order. Once you hold key 103 you know the next guest gets 104. A good hotel uses long, random card codes that tell you nothing about the next one.

How it works (numbered steps):

1. The attacker collects several of their own valid tokens (log in a few times, or sniff a few).
2. They compare them and look for a constant part, a counter, or a timestamp.

3. They test whether the generator is weak (is the token space small? Is the PRNG predictable?).
4. They build a list of candidate tokens for other users.
5. They send the candidates to the server. With no rate limiting, they can try thousands. A hit gives them a victim's session.

Real-world use: A tester signs up for three accounts on an internal portal and sees tokens `A1001`, `A1002`, `A1003`. They script a loop that tries `A1004` to `A1500`. Several return other staff members' pages. The finding is "predictable session tokens" with high severity.

Tools and commands used here: Burp Suite (Sequencer measures token randomness, Intruder automates guessing) and OWASP ZAP. Both are named in section 4. No command line is shown on the slide.

AI angle: An attacker can paste 50 captured tokens into an AI assistant and ask for the pattern. Example prompt: "Here are 50 session tokens from one site. Find the constant part, the counter or timestamp part, and predict the next five." A defender can ask an AI assistant to review the token generation code for weak random functions (for example `rand()` instead of a cryptographically secure generator).

Countermeasures and defense:

- Use a cryptographically secure random generator (CSPRNG) on the server.
- Make tokens long. OWASP suggests at least 64 bits of entropy (randomness) (verify; many frameworks use 128 bits).
- Never build tokens from a counter, a timestamp, an IP address, or user data.
- Add rate limiting and temporary lockout on repeated invalid tokens.
- Regenerate the token after login.

EXAM FOCUS:

- Keywords "identifiable pattern", "counter" -> Sequential tokens.
- Keyword "date and time inside the token" -> Timestamp-based tokens.
- Keywords "few possible values", "guess all tokens" -> Small token space, brute force.
- Keywords "seed", "algorithm" -> Weak/predictable PRNG.
- Keyword "numerous guesses without being blocked" -> Lack of rate limiting.
- Trap: brute force and prediction are different. Prediction uses a pattern to guess smartly. Brute force tries every value.

Memory trick: "STB-PR": Sequential, Timestamp, Brute force (small space), Predictable PRNG, Rate limiting missing.

3.8 Compromising Session IDs Using Man-in-the-Middle / Manipulator-in-the-Middle Attack [page 4]

What it is (simple words): A **man-in-the-middle** (MITM) attack, also written **manipulator-in-the-middle**, intrudes into an existing connection between two systems and intercepts the messages they exchange. The attacker splits the TCP connection into two:

1. **Client-to-attacker connection.**
2. **Attacker-to-server connection.**

After intercepting, the attacker can read, modify, and insert fraudulent data into the communication. In the case of an HTTP transaction, the TCP connection between the client and the server becomes the target. The slide diagram shows the direct victim-to-server link crossed out and two new links through the attacker.

Analogy: A dishonest interpreter at a meeting. Each side thinks they are talking to the other. Everything passes through the interpreter, who can read it, change it, or add new sentences.

How it works (numbered steps):

1. The attacker gets onto the path between client and server (for example with ARP spoofing, a rogue Wi-Fi access point, or a rogue DHCP server).
2. The client opens a TCP connection. The attacker catches it and answers as the server (connection 1).
3. The attacker opens a second TCP connection to the real server and answers as the client (connection 2).
4. Every request and response passes through the attacker. They copy the session cookie from the traffic.
5. They replay the cookie, or edit messages in transit.

Real-world use: At an airport, an attacker sets up a free Wi-Fi hotspot named like the airport network. Travelers connect. The attacker's laptop relays their web traffic and copies every session cookie sent over plain HTTP. The result is access to several webmail accounts.

Tools and commands used here: bettercap, Ettercap, Burp Suite, OWASP ZAP, Hetty, Caido, sslstrip (section 4). ARP spoofing method in section 3.22.

AI angle: None shown on the slide. ADDED CONTEXT: an AI assistant can help a tester write a bettercap caplet (script) for a lab. A defender can use AI-based network detection to spot a host that suddenly appears as the gateway for many machines.

Countermeasures and defense: HTTPS everywhere plus HSTS (section 3.27), Dynamic ARP Inspection and port security on switches (Module 8), DNS over HTTPS

(section 3.28), VPN or IPsec (section 3.29), and teaching users to heed certificate warnings.

EXAM FOCUS:

- Keyword "split the TCP connection into two connections" -> MITM.
- Keyword "client-to-attacker" and "attacker-to-server" -> MITM.
- Terminology: EC-Council now writes "Manipulator-in-the-Middle" next to "Man-in-the-Middle". They mean the same attack.
- Trap: MITM works on the network path. Man-in-the-Browser (section 3.9) works inside the victim's browser, after the data is decrypted.

Memory trick: "Two legs": the attacker has a leg in the client's room and a leg in the server's room.

3.9 Compromising Session IDs Using Man-in-the-Browser / Manipulator-in-the-Browser Attack [page 5]

What it is (simple words): A **man-in-the-browser** (MitB) attack uses a **Trojan horse** (malware that pretends to be useful) to intercept the calls between the browser and its security mechanisms or libraries. The attacker works inside the browser, so the victim's own secure login and HTTPS do not stop it.

Analogy: A dishonest bank clerk standing beside you while you fill in a transfer form. You write "pay 100 to Alice". The clerk quietly changes it to "pay 9,000 to Mallory" before handing it over, then gives you a receipt that still says "100 to Alice".

How it works (14 steps from the slide, grouped):

Infection:

1. The Trojan first infects the OS or an application.
2. The Trojan installs malicious code (extension files) and saves it in the browser configuration.
3. When the user restarts the browser, it loads the malicious extension files.

Arming:

1. The extension files register a handler for every visit to a webpage.
2. When a page loads, the extension matches the URL with a list of targeted sites.

Victim acts normally:

1. The user logs in securely to the website.
2. The extension registers a button event handler for specific page loads.
3. When the user clicks the button, the extension uses the **DOM** (Document Object Model, the browser's live copy of the page) to extract and modify the form data.

Tampering:

1. The browser sends the form and the modified values to the server.
2. The server receives the modified values but cannot tell them from the original.

Cover-up:

1. After the server performs the transaction, a receipt is generated.
2. The browser receives the receipt for the modified transaction.
3. The browser displays the receipt with the original details.
4. The user believes the original transaction was processed without interception.

Real-world use: A customer installs a "free PDF tool" that is really a Trojan. Weeks later, the Trojan's extension waits for the customer to open the bank website. When the customer sends 100 to a supplier, the extension changes the account number. The bank sees a valid, logged-in, correctly authenticated request.

Tools and commands used here: None shown. ADDED CONTEXT: well-known MitB banking Trojan families include Zeus, SpyEye, and Gozi (verify current status).

AI angle: None shown. ADDED CONTEXT: an attacker could use AI to generate obfuscated extension code that evades antivirus. A defender can use AI-based behavior detection to flag a browser extension that reads form fields on banking pages.

Countermeasures and defense (ADDED CONTEXT):

- Keep OS, browser, and antivirus up to date.
- Install only approved extensions; use an enterprise allow-list.
- Verify high-value transactions out-of-band (a phone call or a separate device, called **transaction signing**).
- Use endpoint detection and response (EDR) tools.
- Do not run unknown software.

EXAM FOCUS:

- Keyword "Trojan", "browser extension", "DOM", "form data modified", "receipt shows original" -> Man-in-the-Browser.
- Key trap: MitB defeats HTTPS and many two-factor setups because the change happens inside the browser, before encryption.
- Look-alike: MITM sits on the network; MitB sits in the browser.
- Count: 14 steps. The exam may ask what happens at step 13 (the user sees the original details).

Memory trick: "Browser butler": the butler (extension) reads your letter, changes it, then tells you nothing changed.

3.10 Compromising Session IDs Using Session Replay Attacks [page 5]

What it is (simple words): In a session replay attack, the attacker listens to the conversation between the user and the server and captures the user's **authentication token**. Later, the attacker **replays** the request to the server with the captured token and gains unauthorized access.

Analogy: Recording someone saying their voice password at a phone bank, then playing the recording back to the same bank. The system hears the right words and lets you in.

How it works (5 steps from the slide):

1. The user establishes a connection with the server.
2. The server asks for authentication information as proof of identity.
3. The user sends authentication tokens.
4. The attacker eavesdrops on this conversation and captures the user's authentication tokens.
5. The attacker replays the captured token to the server and gains unauthorized access.

```
User ------ 1. connect ------------------------> Server
User <----- 2. send proof of identity ---------- Server
User ------ 3. auth token ----+-----------------> Server
                              |
                         4. copy          (Attacker listens)
                              |
Attacker ---- 5. same token ------------------> Server
```

Real-world use: An employee logs into an internal app over plain HTTP. A tester on the same network captures the `Cookie:` header with Wireshark, then pastes the same value into a browser cookie editor. The app opens the employee's account. The tester never sent a password.

Tools and commands used here: Wireshark or tcpdump to capture; Burp Suite Repeater or a browser cookie editor to replay (ADDED CONTEXT).

AI angle: None shown. ADDED CONTEXT: a defender can ask an AI assistant to write a log query that flags the same token used from two countries within minutes.

Countermeasures and defense (ADDED CONTEXT):

- Encrypt all traffic with HTTPS.
- Use short token lifetimes and expire tokens at logout.
- Add one-time values: a **nonce** (number used once) or a timestamp in each request, so an old captured request is rejected.
- Bind the token to the TLS connection (Token Binding, section 3.27) or to the client device.

- Detect the same token from two different IP addresses.

EXAM FOCUS:

- Keyword "captures the authentication token" and "replays" -> Session replay.
- Trap: replay does not need to break the victim's connection. The victim can stay logged in. This differs from the active TCP hijack in section 3.17, which knocks the victim off.
- Look-alike: replay reuses a token the attacker copied; fixation (3.11) makes the victim use a token the attacker already knew.

Memory trick: "Record and replay", like a tape recorder.

3.11 Compromising Session IDs Using Session Fixation [page 6]

What it is (simple words): Session fixation is an attack that lets an attacker hijack a valid user session. The attacker lures a user to authenticate with a **known** session ID, then hijacks the user-validated session using that known ID. The attacker has to provide a legitimate web application session ID and lure the victim's browser into using it. The attacker exploits a server vulnerability: the server allows a user to keep using a fixed SID (it does not issue a new one at login).

Analogy: A thief hands you a hotel key card he already copied and says "use this one". You walk in, put your valuables in the room safe, and he walks in later with his copy.

Three techniques to deliver the fixed SID (from the slide):

1. **Session token in the URL argument.**
2. **Session token in a hidden form field.**
3. **Session ID in a cookie.**

How it works (4 steps from the slide diagram):

1. The attacker gets a valid SID from the site and emails the victim a link that carries it. Slide example: "Hi John, check this out, here are cool new festival offers from our bank, `http://citibank.com/?SID=0D6441FEA4496C2`. Thanks, Customer Care, Citibank".
2. The victim clicks the link and logs in on the vulnerable server. The browser sends `POST account.php` with `Cookie: 0D6441FEA4496C2`.
3. The attacker uses the same SID (`sessionid=0D6441FEA4496C2`) and connects to the server.
4. The attacker sends `POST account.php` with the same cookie. The server thinks the attacker is the logged-in victim.

```
Attacker ---(1) email link with SID=0D64...---> Victim
Victim   ---(2) logs in with that SID --------> Web Server
Attacker ---(3) opens site with same SID -----> Web Server
Attacker ---(4) POST account.php, same cookie > Web Server
                 Server sees one valid logged-in session
```

Real-world use: A phishing email to a bank customer carries a link with a preset session ID. The customer logs in. Because the site never issues a fresh ID at login, the attacker's copy of the ID is now an authenticated session.

Tools and commands used here: Burp Suite or OWASP ZAP to test whether a site changes the SID after login; browser developer tools to compare the cookie before and after login.

AI angle: An attacker can use an AI writing tool to produce a fluent, convincing bank email in many languages. Example prompt (for an authorized phishing awareness test): "Write a short customer-care email from a bank announcing a seasonal offer, with a placeholder for a link." A defender can use AI email filters to flag links that carry a `SID=` parameter.

Countermeasures and defense:

- **Regenerate the session ID immediately after a successful login.** This is the main fix.
- Accept only server-generated session IDs. Reject any SID the server did not issue.
- Do not accept session IDs in the URL or in hidden fields. Use cookies with `Secure` and `HttpOnly`.
- Destroy the old session at logout and on timeout.

EXAM FOCUS:

- Keywords "lure the user to authenticate with a known session ID", "fixed SID" -> Session fixation.
- The attack happens BEFORE login: the attacker knows the SID first, then the victim authenticates.
- Must-memorize: URL argument, hidden form field, cookie.
- Trap: session fixation (attacker sets the SID, then waits) vs session hijacking by sniffing (attacker steals the SID after it is issued).
- Best fix: regenerate the SID at login.

Memory trick: "Fix it first, then wait": the attacker fixes the SID first, the victim logs in, the attacker walks in.

3.12 Session Hijacking Using Proxy Servers [page 6]

What it is (simple words): The attacker lures the victim to click a bogus link that looks legitimate but redirects the user to the attacker's server. The attacker forwards

the victim's requests to the real server and serves as a **proxy** (a middleman server that passes requests on behalf of someone else) for the whole transaction. The attacker captures the session's information during the exchange between the legitimate server and the user.

Analogy: A fake receptionist who sits in front of the real reception desk. Visitors talk to the fake one, who passes every question to the real desk and every answer back, while quietly writing down their names and badge numbers.

How it works (4 steps from the slide):

1. The attacker sends an email with a malicious link. Slide example: the text "Click here for a 50% discount at Amazon.com!" with an HTML link `<a href="http://reallybadguys.com/gotcha.php">`.
2. When the victim clicks, the link redirects the user to the attacker's server (`reallybadguys.com`).
3. The attacker's server forwards the request to the legitimate server (`amazon.com`) on behalf of the victim.
4. The attacker's server serves as a proxy for the entire transaction and captures the session information.

```
Attacker --1. email with link------------> Victim
Victim   --2. clicks, goes to bad site----> Attacker Srv
Attacker Srv --3. forwards request-------> amazon.com
Attacker Srv <--4. relays both ways------> amazon.com
```

Real-world use (ADDED CONTEXT): Modern "reverse proxy phishing" kits such as Evilginx and Modlishka do exactly this. The victim sees a real login page (relayed live), enters a password and passes multi-factor authentication, and the proxy steals the resulting session cookie. The attacker then imports the cookie and skips MFA. Verify current kit names.

Tools and commands used here: None shown on the slide. Related tooling: Burp Suite and mitmproxy (legitimate proxies, section 4), Evilginx and Modlishka (adversary-in-the-middle phishing, ADDED CONTEXT).

AI angle: AI-written phishing text and AI-cloned login pages make this lure easier to scale. A defender can use an AI classifier on URLs and on page content to flag look-alike domains.

Countermeasures and defense:

- Check the address bar before typing credentials.
- Use phishing-resistant MFA such as FIDO2/WebAuthn security keys (ADDED CONTEXT). They are tied to the real domain, so a fake domain gets no valid response.
- Use secure email gateways and URL rewriting.
- Short session lifetimes and device-bound sessions.

- User awareness training.

EXAM FOCUS:

- Keywords "bogus link", "attacker server serves as a proxy for the entire transaction" -> Session hijacking using proxy servers.
- Trap: this is not the same as MITM on the network. Here the victim is tricked by a link and does not need the attacker to be on the victim's LAN.
- Look-alike: proxy hijacking relays traffic live; session fixation presets an ID; session donation gives the victim the attacker's ID.

Memory trick: "Fake front desk": the relay desk sees everything.

3.13 Session Hijacking Using the CRIME Attack (ADDED CONTEXT) [slide 16 not in screenshots]

> **ADDED CONTEXT:** Slide 16 is missing from your screenshots. It sits between "Proxy Servers" (slide 15) and "Forbidden Attack" (slide 17). It may be the CRIME attack, which older EC-Council decks place at this point. This is a best guess. Verify against the video.

What it is (simple words): CRIME stands for Compression Ratio Info-leak Made Easy. It is an attack on TLS/SSL (the encryption used by HTTPS) when **compression** is turned on. The attacker learns secret data such as a session cookie by watching how the size of encrypted messages changes.

Analogy: You cannot read a sealed envelope, but you can weigh it. If you add a guess to the letter and the envelope gets lighter (because the letter compressed better), your guess matched a secret already inside.

How it works (numbered steps):

1. The attacker is on the network path (MITM) and can inject text into requests the victim's browser sends, for example through a script in another tab.
2. The victim's browser sends a request that contains the secret cookie plus the attacker's guess for it.
3. The request is compressed, then encrypted. If the guess matches part of the real cookie, compression shrinks the message more.
4. The attacker watches the encrypted message length. A shorter length means a correct guess.
5. Repeat one character at a time until the cookie is recovered. The attacker then hijacks the session with it.

Real-world use: Researchers showed CRIME against TLS compression in 2012. Browsers and servers then disabled TLS-level compression. Verify details. A related

attack, **BREACH**, targets HTTP-level (gzip) response compression instead and is harder to remove completely.

Tools and commands used here: None needed on the exam.

Countermeasures and defense: Disable TLS compression; disable or protect HTTP compression for pages that mix secrets and user input; randomize or mask secrets per response; use SameSite cookies and CSRF tokens.

EXAM FOCUS:

- Keyword "compression" plus "session cookie" -> CRIME.
- Keyword "TLS compression" -> CRIME; "HTTP compression" -> BREACH.
- Trap: CRIME does not break encryption. It leaks length information.

Memory trick: "Crime scene weigh-in": CRIME = Compression leaks size.

3.14 Session Hijacking Using the Forbidden Attack [page 7]

What it is (simple words): A **Forbidden attack** is a type of man-in-the-middle attack used to hijack **HTTPS sessions**. It exploits the **reuse of a cryptographic nonce** during the TLS handshake (as the slide states). A **nonce** is a "number used once": a value that must never repeat for the same key. After hijacking the HTTPS session, the attacker injects malicious code and forged content that prompts the victim to disclose sensitive information such as bank account numbers, passwords, and social security numbers.

Analogy: A one-time stamp that is supposed to be used only once. If a shop reuses the same stamp, a forger can copy it and stamp fake receipts that look genuine.

How it works (6 steps from the slide diagram):

1. The victim and web server perform a TLS handshake to set up the HTTPS connection.
2. The attacker sniffs the traffic.
3. The traffic between the server and the victim now flows through the attacker.
4. The attacker uses the reused nonce to hijack the session.
5. The attacker injects malicious code and forged content into the session.
6. The victim reveals sensitive information.

> **ADDED CONTEXT:** The name "Forbidden attack" comes from research on **AES-GCM**, a common TLS encryption mode. If a server repeats a nonce with the same key, an attacker can forge messages and inject content into the encrypted session. Researchers presented this at WOOT 2016 and found 184 HTTPS servers repeating nonces and over 70,000 using random nonces that risk repeating. Technically the reuse happens in the encrypted record data after the handshake. The slide says "during the TLS handshake". Use the slide wording for the exam. Verify.

Real-world use: A tester proves that a misconfigured HTTPS server accepts forged messages after a nonce repeats. The report states that HTTPS alone did not protect the session, and recommends upgrading the TLS library and cipher settings.

Tools and commands used here: None shown. Related: Wireshark to observe TLS records; TLS test scanners (ADDED CONTEXT).

AI angle: None shown.

Countermeasures and defense (ADDED CONTEXT): Patch TLS libraries; use implementations that generate nonces with a counter, not random values; prefer TLS 1.3; disable weak cipher suites; monitor certificate and TLS configuration with scanners.

EXAM FOCUS:

- Keywords "cryptographic nonce", "reuse", "HTTPS sessions" -> Forbidden attack.
- Trap: it targets HTTPS, so "use HTTPS" alone is not the full answer here. Correct configuration of TLS matters.
- Look-alike: CRIME leaks data through compression; Forbidden attack forges data through nonce reuse.

Memory trick: "Forbidden = nonce reused": a nonce is forbidden from being used twice.

3.15 Session Hijacking Using Session Donation Attack [page 7]

What it is (simple words): In a **session donation attack**, the attacker **donates** his or her own session identifier (SID) to the target user. The attacker first obtains a valid SID by logging into a service, and later feeds the same SID to the target user. This SID links the target user back to the attacker's account page without any information to the victim.

Analogy: You give a friend a free gift card that is already linked to your own account. Everything your friend buys or saves lands in your account history.

How it works (5 steps from the slide):

1. The attacker logs into a service and establishes a legitimate connection.
2. The web server issues a SID (slide example `SID=0D6441FEA4496C2`).
3. The attacker donates this SID and lures the victim to click it to access the website. Slide email: "Hi Bob, check this out, here are new seasonal offers from our bank, `http://citibank.com/?SID=0D6441FEA4496C2`".
4. The victim clicks the link, enters his or her details, and saves them.
5. The attacker logs in and acquires the victim's information.

```
Attacker --1. login, 2. server issues SID-------------> Web Server
Attacker --3. link with attacker's SID----------------> Victim
Victim   --4. enters details into attacker's session---> Web Server
Attacker --5. logs in, reads victim's data------------> Web Server
```

Real-world use: A victim is tricked into saving card details inside a shopping site session that actually belongs to the attacker. The attacker later logs into his own account and sees the saved card.

Tools and commands used here: None shown.

AI angle: AI-written lure emails (same as section 3.11).

Countermeasures and defense: Do not accept session IDs from URLs; reject login links with a preset SID; make users re-authenticate at sensitive steps; use CSRF tokens; show "you are logged in as ..." with clear account identity so victims notice a foreign account (ADDED CONTEXT).

EXAM FOCUS:

- Keywords "donates his/her own SID", "links a target user back to the attacker's account" -> Session donation.
- Main difference from fixation: in fixation the attacker WAITS for the victim to log in and then uses the victim's session. In donation the attacker is already logged in, and the victim puts data into the ATTACKER's account.
- Look-alike table row is in section 7.2.

Memory trick: "Donation = I give you MY ticket." The ticket goes to my account, not yours.

3.16 Network-Level Session Hijacking [page 8]

> **ADDED CONTEXT:** Slide 19 is missing from your screenshots. It sits just before this topic and is probably a divider slide for the network-level section.

What it is (simple words): Network-level hijacking relies on hijacking the **transport** protocols (TCP, UDP) and **Internet** protocols (IP, ICMP, ARP) that web applications in the application layer depend on. By attacking network-level sessions, the attacker gathers critical information (for example sequence numbers or session cookies in cleartext) that is then used to attack the application-level sessions.

Analogy: An application-level attack steals the key to the front door. A network-level attack takes over the road the visitor is driving on, so the attacker can change the visitor's destination or read the mail in the car.

The six network-level techniques on the slide:

1. Blind hijacking (section 3.20).

2. UDP hijacking (section 3.21).
3. TCP/IP hijacking (section 3.17).
4. RST hijacking (section 3.19).
5. Man-in-the-middle: packet sniffer (section 3.22).
6. IP spoofing: source routed packets (section 3.18).

The slide order is: Blind, UDP, TCP/IP, RST, MITM packet sniffer, IP spoofing. This guide follows the order of the screenshots for the detailed slides (TCP/IP first, then source routing, RST, blind and UDP, MITM). The exam only cares about the six names.

How it works (numbered steps):

1. The attacker gets on the network path (same subnet, or a position that sees the traffic).
2. They sniff to learn IP addresses, ports, and sequence numbers.
3. They forge or inject packets with the right numbers so the server accepts them.
4. They knock the victim out, or race the victim, so only the attacker's packets are accepted.
5. They use the position to read, change, or add data to the session.

Real-world use: On a flat office network, a tester captures a Telnet session between an admin and a legacy server, then injects packets with the correct sequence numbers to run a command as the admin.

Tools and commands used here: Wireshark, tcpdump, Colasoft Packet Builder, hping3 and Scapy (ADDED CONTEXT), bettercap, Ettercap (section 4 and 5).

Countermeasures and defense: Encryption at the network layer (IPsec, VPN), encrypted protocols (SSH instead of Telnet), switched networks, Dynamic ARP Inspection, randomized initial sequence numbers, disabling source routing, ingress/egress filtering (section 3.25).

EXAM FOCUS:

- Keyword "transport and Internet protocols" -> Network-level hijacking.
- Count: six techniques.
- Trap: network-level hijacking needs access to the traffic path. Application-level attacks such as session fixation need only a lure.

Memory trick: "B-U-T-R-M-I": Blind, UDP, TCP/IP, RST, MITM, IP spoofing.

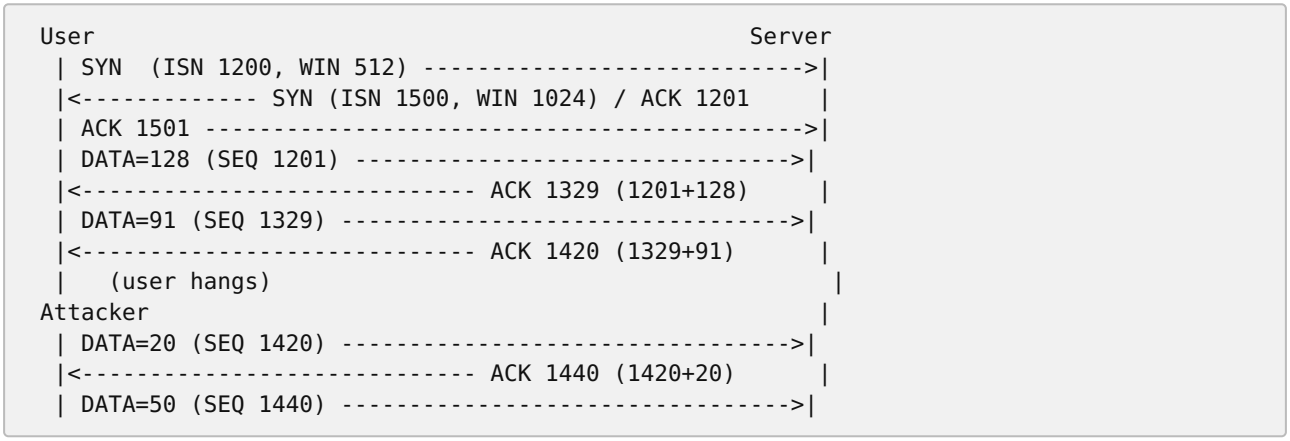
3.17 TCP/IP Hijacking [page 8]

What it is (simple words): TCP/IP hijacking uses **spoofed packets** (packets with a forged source address) to seize control of a connection between a victim and a target machine. The victim's connection hangs, and the attacker can then communicate with the host machine as if the attacker is the victim. To launch the

attack, the attacker must be on the **same network** as the victim. The target server and the victim machine can be anywhere.

Analogy: Two people are exchanging numbered letters. Letter 1, 2, 3 and so on. You intercept the mail carrier, learn the next number is 4, and mail your own letter numbered 4 to the recipient before the real sender does. The recipient accepts yours and rejects the real letter 4 as a duplicate.

How it works (reading the slide diagram):



Numbered steps:

1. The user and server complete the three-way handshake. The client picks ISN 1200 with window 512. The server picks ISN 1500 with window 1024 and acknowledges 1201. The client acknowledges 1501.
2. The user sends 128 bytes starting at SEQ 1201. The server answers ACK 1329 (1201 + 128).
3. The user sends 91 bytes at SEQ 1329. The server answers ACK 1420 (1329 + 91).
4. Before the user sends the next data packet, the attacker predicts the next sequence number (1420) and sends data to the server with that number (20 bytes, SEQ 1420).
5. The server accepts it and answers ACK 1440 (1420 + 20). The attacker sends the next packet at SEQ 1440 (50 bytes).
6. The slide note says: this leads to the establishment of the connection between the attacker and the server. The attacker now owns the session.

What happens to the victim (ADDED CONTEXT): The victim still thinks the next sequence number is 1420. The server now expects 1440. The two sides disagree (they are **desynchronized**). Each side keeps sending ACKs and resending packets, which can cause an "ACK storm" of useless traffic. The victim's session hangs, which is the "victim's connection hangs" line on the slide.

Real-world use: A historic example is the 1994 attack on a computer security researcher's network, which combined IP spoofing with sequence number prediction

(verify details). A modern lab version uses a tool that sniffs a Telnet or FTP session, desynchronizes the client, and takes the shell.

Tools and commands used here: Wireshark and tcpdump (watch sequence numbers), hping3 and Scapy for crafted packets, Ettercap (section 4 and 5). Older hijack tools named in earlier EC-Council versions: Hunt, Juggernaut, Shijack, T-Sight, Paros HTTP Hijacker (ADDED CONTEXT, verify).

AI angle: None shown. ADDED CONTEXT: an AI assistant can explain a packet capture. Example prompt: "Here is a tcpdump excerpt. List each packet, show if SEQ and ACK numbers line up, and point out any packet that breaks the sequence."

Countermeasures and defense:

- Encrypt the session end to end (SSH, TLS, IPsec). Encryption stops the attacker from running useful commands even if the TCP segment is accepted.
- Use randomized initial sequence numbers (RFC 6528; verify).
- Use a switched network and Dynamic ARP Inspection so the attacker cannot sniff.
- Use packet-filtering firewalls that drop packets claiming to come from inside but arriving from outside.
- Use IDS rules that detect ACK storms and duplicate sequence numbers.

EXAM FOCUS:

- Keywords "predicts the next sequence number", "same network as the victim" -> TCP/IP hijacking.
- Must-know: the attacker has to be on the same network as the victim. The server and victim may be far away.
- Arithmetic: next ACK = last SEQ + length of data.
- Trap: the question may say "the attacker is on the Internet" with the server. The attacker must still share the victim's network segment.
- Look-alike: TCP/IP hijacking injects data into a live session; RST hijacking kills a session (section 3.19).

Memory trick: "Guess the next number, speak first." The server hears the attacker first.

3.18 IP Spoofing: Source Routed Packets [page 9]

What it is (simple words): IP spoofing means forging the source IP address in a packet. **Source routing** is an IP option that lets the sender choose the path the packet takes. The **packet source routing technique** is used to gain unauthorized access to a computer with the help of a trusted host's IP address.

Analogy: You write a return address of your friend's house on a letter, and you also write a route on the envelope: "deliver this by way of my friend's house, then to my real mailbox". The reply is forced through the friend's house and arrives at your hands.

How it works (5 steps from the slide):

1. The packet source routing technique is used for gaining unauthorized access to a computer with the help of a trusted host's IP address.
2. The attacker spoofs the host's IP address so that the server managing a session with the host accepts the packets from the attacker.
3. When the session is established, the attacker injects forged packets before the host responds to the server.
4. The original packet from the host is lost, because the server receives a packet with a sequence number already used by the attacker.
5. The packets from the attacker are source-routed through the host, with the destination IP specified by the attacker.

ADDED CONTEXT: why source routing matters. In normal IP spoofing, the reply goes to the real owner of the forged address, so the attacker never sees it (this is the "blind" case, section 3.20). Source routing fixes that. IP has two options (verify): **Loose Source Route** (LSRR, the packet must pass through listed routers but may take other hops) and **Strict Source Route** (SSRR, the packet must follow the listed path exactly). The attacker lists their own address in the route, so replies travel back through them.

Real-world use: An old UNIX network trusts connections from a management host by IP address (rsh/rlogin style trust). An attacker forges the management host's IP and a source route that sends the reply to the attacker. The server accepts the session as trusted.

Tools and commands used here: hping3 (supports IP options and spoofed source, ADDED CONTEXT), Scapy, Wireshark to inspect IP options. Details in sections 4 and 5.

Countermeasures and defense:

- Disable source routing on routers and hosts (Cisco `no ip source-route`; Linux `net.ipv4.conf.all.accept_source_route = 0`).
- Do not trust IP addresses alone for authentication (stop using `.rhosts` trust).
- Ingress and egress filtering: block packets that claim a source address that cannot come from that interface.
- Use encrypted, authenticated protocols.

EXAM FOCUS:

- Keywords "trusted host's IP address", "source-routed", "destination IP specified by the attacker" -> IP spoofing: source routed packets.
- Trap: source routing is the trick that lets a spoofing attacker SEE the replies. Without it, IP spoofing is blind.
- Countermeasure keyword "disable source routing".

Memory trick: "Route me through you": the attacker writes their own stop on the route.

3.19 RST Hijacking [page 9]

What it is (simple words): RST hijacking injects an authentic-looking **reset (RST) packet** using a spoofed source address and a predicted acknowledgment number. A TCP RST flag tells the other side "abort this connection now". A hacker can reset a victim's connection if the packet uses an accurate acknowledgment number. The victim believes the source sent the reset packet and resets (closes) the connection.

Analogy: A forged "Meeting cancelled, everyone go home" note signed with the organizer's name. If the note looks genuine, people leave.

How it works (4 steps from the slide diagram):

```
Victim 192.168.0.100              Server 192.168.0.200
  | 01 data  SRC .100 DST .200       |
  |    SEQ 1429775000 ACK 1250510000 |
  |    LEN 24 ---------------------->|
  |<-------------------- 02 reply    |
  |   SRC .200 DST .100              |
  |   SEQ 1250510000 ACK 1429725024  |
  |   LEN 167                        |
  |                                  |
  |<--- 03 Attacker spoofs server IP |
  |     04 predicts ACK  sends RST   |
  |                                  |
```

1. The victim machine (192.168.0.100) sends a data packet to the server (192.168.0.200). Slide values: SEQ 1429775000, ACK 1250510000, LEN 24.
2. The server replies. Slide values: SEQ 1250510000, ACK 1429725024 (see the note below), LEN 167.
3. The attacker spoofs the IP address of the server machine.
4. The attacker predicts the ACK number and sends an RST packet to reset the connection. The victim thinks the server sent it and closes the session.

> **TRAP:** [UNCLEAR on page 9] The ACK number in the server's reply reads 1429725024 on the slide. The sum SEQ 1429775000 + LEN 24 gives 1429775024. This may be a typo on the slide or a misreading of blurry text. The idea (ACK = previous SEQ + length) is not affected.

ADDED CONTEXT: which number really has to be right. For a forged RST to be accepted, its SEQ number must fall inside the victim's receive window, and ideally equal the number the victim expects next from the server. The victim last announced that number in its ACK field. This is why the attacker must watch (or predict) the acknowledgment values. The slide simply says "an accurate acknowledgment number". Newer TCP stacks (RFC 5961, verify) answer a nearly-correct RST with a challenge ACK to make blind resets harder.

Real-world use: Some networks have been reported to cut unwanted connections by injecting forged RST packets (verify examples). A tester in a lab uses RST injection to prove that an unencrypted long-lived session (for example a database link) can be killed from a neighbor machine.

Tools and commands used here: The slide names **Colasoft Packet Builder** (a Windows tool that builds custom packets) as a packet crafting tool and **tcpdump** as a TCP/IP analysis tool. ADDED CONTEXT: hping3 and Scapy can also craft RST packets in a lab (section 5).

AI angle: None shown. ADDED CONTEXT: an AI assistant can read a capture and explain why a connection ended. Example prompt: "In this tcpdump output, which packet carries the RST flag, and does its sequence number match what the other side expected?"

Countermeasures and defense (ADDED CONTEXT):

- Network-layer authentication and encryption (IPsec) or TCP-AO (TCP Authentication Option, RFC 5925, verify) protect the TCP header itself. TLS alone does not, because the TCP header is outside the TLS payload.
- Use switched networks and Dynamic ARP Inspection so the attacker cannot see the sequence numbers.
- Use randomized initial sequence numbers and modern TCP stacks (RFC 5961 challenge ACK).
- Use IDS or IPS rules that flag RST packets with unusual TTL or from unexpected sources.

EXAM FOCUS:

- Keywords "reset (RST) packet", "spoofed source address", "predicting the acknowledgment number" -> RST hijacking.
- Tools named: Colasoft Packet Builder (craft packets), tcpdump (analyze).
- Result of the attack: the victim's connection is torn down. The attacker does not necessarily gain control. Compare this with TCP/IP hijacking, which injects data and takes control.
- Trap: RST hijacking is a TCP attack. It does not apply to UDP, which has no reset flag.

Memory trick: "RST = Reset the Session Terminally."

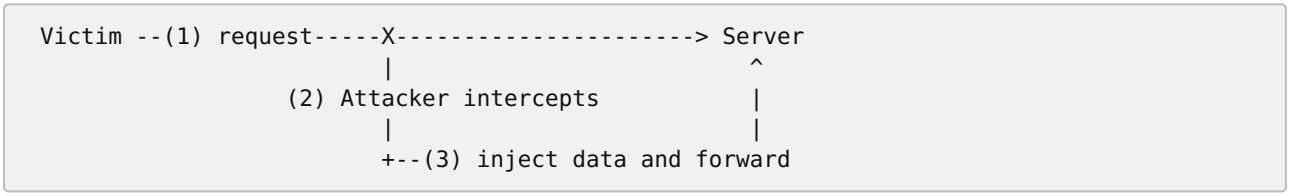
3.20 Blind Hijacking [page 10]

What it is (simple words): With **blind hijacking**, the attacker can inject malicious data or commands into the intercepted communication in a TCP session even if source routing is disabled. The attacker can send the data or commands but has **no access to see the response**.

Analogy: Shouting instructions through a wall to someone in the next room. They carry out your orders, but you cannot hear or see what they answer.

How it works (numbered steps, from the slide diagram):

1. The victim sends a request to the server.
2. The attacker intercepts the request.
3. The attacker injects malicious data or commands into the request and forwards it to the server.
4. The server acts on it. The reply goes to the victim, not the attacker, so the attacker never sees the result.



ADDED CONTEXT: classic example. Because the attacker is blind, they choose commands whose effect does not need to be seen. A well-known textbook example is adding a trust entry (such as `+ +` in a `.rhosts` file) so that a later normal login from anywhere succeeds. Another is creating a new user account. Verify as an example only.

Real-world use: A tester injects a command into a lab Telnet session to create an account named `testuser`, then logs in with it from a second window. They never saw the first command's output, but the account proves the injection worked.

Tools and commands used here: Packet crafting tools (hping3, Scapy), sniffers to guess sequence numbers.

Countermeasures and defense: Encrypt sessions; randomize initial sequence numbers; avoid unauthenticated plain text protocols (Telnet, rsh, FTP); filter spoofed packets.

EXAM FOCUS:

- Keywords "no access to see the response", "even if source routing is disabled" -> Blind hijacking.
- Trap: the attacker still must guess sequence numbers correctly. "Blind" refers to not seeing the replies, not to guessing at random.
- Look-alike: source-routed spoofing (3.18) lets the attacker SEE replies; blind hijacking does not.

Memory trick: "Blind = mouth, no eyes."

3.21 UDP Hijacking [page 10]

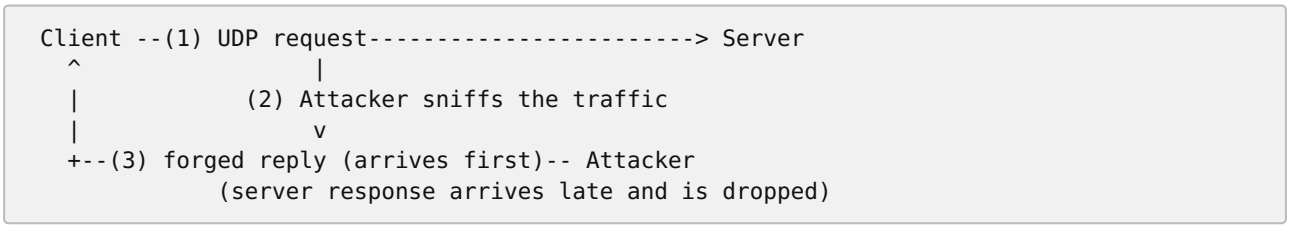
What it is (simple words): UDP hijacking is a network-level session hijacking where the attacker sends a **forged server reply** to a victim's UDP request before the intended server replies to it. The attacker uses a man-in-the-middle attack to intercept the server's response to the client and sends a forged reply.

Background: UDP (User Datagram Protocol) is connectionless. It has no handshake, no sequence numbers, and no reset. It is used for DNS, DHCP, SNMP, TFTP, NTP, and streaming (ADDED CONTEXT). That makes it easy to forge a reply: the attacker only needs to be faster and to match the addresses, ports, and any application-level request ID.

Analogy: A quiz show where the first person to shout an answer wins. The attacker shouts a fake answer before the real expert can speak, and the host accepts it.

How it works (numbered steps, from the slide diagram):

1. The client sends a UDP request to the server.
2. The attacker sniffs the traffic and sees the request.
3. The attacker forges a reply before the real server responds.
4. The client accepts the first reply it receives.
5. The real server's response (crossed out on the slide) arrives later and is ignored or dropped.



Real-world use: A forged DNS reply sends a victim to the wrong IP address (DNS spoofing). The victim then opens the attacker's copy of a banking site.

Tools and commands used here: None named on the slide. ADDED CONTEXT: Ettercap, bettercap, and Scapy can forge UDP replies in a lab.

Countermeasures and defense: Use DNSSEC, random source ports and random transaction IDs; use authenticated and encrypted transports (DNS over HTTPS, DTLS, QUIC); use application-level validation.

EXAM FOCUS:

- Keywords "forged server reply", "before the intended server" -> UDP hijacking.
- Trap: UDP has no sequence numbers, so the sequence number prediction steps of TCP hijacking do not apply here.
- Look-alike: UDP hijacking is a race; TCP/IP hijacking is a number guess.

Memory trick: "UDP = first voice wins."

3.22 MiTM Attack Using Forged ICMP and ARP Spoofing [page 10]

What it is (simple words): In this attack, the **packet sniffer** is used as an **interface** between the client and the server. The attacker changes the **default gateway** of the client's machine and attempts to reroute packets. The packets between client and server are routed through the hijacker's host using two techniques:

- **Forged ICMP (Internet Control Message Protocol):** ICMP is an extension of IP used to send error messages. The attacker sends forged ICMP messages to fool the client and server.
- **ARP (Address Resolution Protocol) spoofing:** ARP maps network layer addresses (IP addresses) to link layer addresses (MAC addresses). The attacker forges ARP replies so a victim thinks the attacker's MAC address belongs to the gateway or to the server.

Analogy: A road sign changed by a prankster. Forged ICMP is a fake "detour ahead" sign. ARP spoofing is swapping the address plate on a neighbor's door so the postman delivers to the wrong house.

How it works (numbered steps):

1. The attacker joins the same subnet as the victim and the gateway.
2. The attacker sends forged ARP replies to the victim: "the gateway's IP has my MAC". They send forged replies to the gateway too: "the victim's IP has my MAC".
3. The victim's traffic now goes to the attacker. The attacker forwards it to the real gateway so the connection still works (IP forwarding on).
4. As an alternative or addition, the attacker sends a forged ICMP redirect that tells the victim "use this other router for that destination" (ADDED CONTEXT: ICMP type 5, Redirect).
5. The attacker's sniffer reads every packet, copies session cookies, and can inject new packets.

```
Victim <--ARP lie--> Attacker <--ARP lie--> Gateway --> Server
  |                     ^                     ^
  +--- traffic ---------+----- forwarded ------+
```

Real-world use: On a hotel LAN, an attacker ARP-poisons a guest and the gateway, then runs a sniffer. All of the guest's unencrypted web sessions pass through the attacker's laptop.

Tools and commands used here: arpspoof, Ettercap, bettercap (ADDED CONTEXT, commands in section 5); Wireshark to capture.

AI angle: None shown. ADDED CONTEXT: an AI tool can help summarize a capture file and highlight duplicate IP-to-MAC answers.

Countermeasures and defense:

- Dynamic ARP Inspection (DAI) and DHCP snooping on switches (Module 8).
- Static ARP entries for critical hosts such as the gateway.
- Use ARPwatch or an IDS to watch for ARP cache poisoning (this is point 12 in the slide 29 checklist, section 3.25).
- Ignore ICMP redirects (`no ip redirects` on Cisco; on Linux set `net.ipv4.conf.all.accept_redirects = 0`).
- Encrypt traffic (HTTPS, SSH, IPsec).

EXAM FOCUS:

- Keywords "changes the default gateway of the client", "routed through the hijacker's host", "ARP spoofing" -> this slide.
- Two techniques: forged ICMP and ARP spoofing.
- Trap: ARP works at Layer 2 (Data Link). ICMP works at Layer 3 (Network). The slide places both under the MITM theme.
- Defense keyword: "monitor ARP cache poisoning" -> ARPwatch or IDS.

Memory trick: "I-A": ICMP fakes the route, ARP fakes the address.

3.23 PetitPotam Hijacking [page 11]

What it is (simple words): In a **PetitPotam** attack, an attacker forces a **domain controller** (DC, the server that holds Active Directory and checks logins in a Windows network) to start authentication to the attacker's server. The attacker uses Microsoft's **Encrypting File System Remote Protocol (MS-EFSRPC)** API for the authentication session hijacking. Then the attacker **relays** the **NTLM** authentication sent by the domain controller to the **Active Directory Certificate Services (AD CS)** server and generates a **certificate** that gives admin-level privileges.

Key words explained:

- **NTLM** (NT LAN Manager): an older Windows challenge-response authentication protocol. The password is never sent, but the response can be relayed to another server.
- **Relay:** passing someone's authentication message on to a different server, which accepts it.
- **Coercion:** forcing a machine to authenticate to a place you choose.
- **AD CS:** the Windows server role that issues digital certificates. A certificate for a domain controller can be used to log in as that controller.
- **SMB** (Server Message Block): the Windows file-sharing protocol, used here as the channel the DC connects to (default TCP 445).

Analogy: Tricking a bank manager into phoning your fake office "to verify an account". The manager reads out a secret verification code. You pass the same code

to the bank's certificate department, which trusts it and prints you a master key card in the manager's name.

How it works (5 steps from the slide diagram):

1. The attacker (running an SMB server) has a legitimate user's NTLM credentials. (Slide wording. See the note below.)
2. The attacker provides these NTLM credentials to the domain controller and calls the MS-EFSRPC function `EfsRpcOpenFileRaw`, passing the attacker's SMB server address as the "file" path.
3. The domain controller connects to the attacker's SMB server and sends its NTLM authentication.
4. The attacker replays (relays) the NTLM authentication to the AD CS server.
5. The AD CS server accepts it and generates an AD certificate for the domain controller account. The attacker now has admin-level privileges.

```
Attacker (SMB server)                    Domain Controller
  | 02 EfsRpcOpenFileRaw(attacker SMB) ------>|
  |<----------- 03 NTLM authentication --------|
  |                                           AD CS Server
  | 04 NTLM authentication replay ------------>|
  |<----------- 05 AD certificate generated ---|
```

ADDED CONTEXT: what the certificate allows. With a certificate issued for the domain controller's machine account, the attacker can request a Kerberos ticket (using PKINIT, certificate-based Kerberos login) and then copy password data from Active Directory (a "DCSync"). That gives domain-wide control. Verify the detailed chain with Microsoft guidance.

ADDED CONTEXT: note on credentials. The slide shows the attacker "having legitimate user's NTLM credentials". Public write-ups of the first PetitPotam releases (2021) said the EFSRPC call could be made without credentials against unpatched systems. Later patches reduced that and many cases now need a domain account. Verify against current Microsoft guidance (CVE-2021-36942 and KB5005413).

Real-world use: During an internal penetration test, a tester with a low-privileged domain account finds AD CS web enrollment still accepting NTLM. They coerce a domain controller, relay to AD CS, get a certificate, and show domain admin access within minutes. The report recommends disabling NTLM relay paths.

Tools and commands used here: None shown on the slide. ADDED CONTEXT: the public PetitPotam proof-of-concept script, Impacket `ntlmrelayx`, and Certipy are the usual lab tools. Use only in an authorized lab.

AI angle: None shown. ADDED CONTEXT: defenders can feed AD CS configuration exports to an AI assistant and ask it to flag templates and endpoints that still accept NTLM.

Countermeasures and defense (ADDED CONTEXT, per Microsoft KB5005413):

- Disable NTLM authentication on the AD CS server and domain controllers where possible.
- Enable **Extended Protection for Authentication (EPA)** on AD CS web enrollment and require HTTPS.
- Disable the web enrollment role if not needed.
- Require SMB signing and apply current Windows security patches.
- Monitor for unusual certificate requests for domain controller accounts.

EXAM FOCUS:

- Keywords "domain controller is forced to authenticate", "MS-EFSRPC", "relays NTLM to AD CS", "generates a certificate" -> PetitPotam.
- It is an NTLM relay variant of session hijacking: the authentication session is taken over.
- Trap: PetitPotam does not crack passwords. It relays the authentication and asks for a certificate.
- Number to remember: 5 steps.

Memory trick: "Pet the Potam, get the Pass": coax (coerce) the DC, pass the NTLM, print a certificate.

3.24 Session Hijacking Tools [page 11]

What it is (simple words): The slide shows two main tools and lists five more.

- **Hetty:** an HTTP toolkit that lets attackers perform a machine-in-the-middle (MITM) HTTP **proxy** attack using logs and advanced search. The screenshot shows version 0.7.0 with a "Proxy logs" list of HTTP requests (GET requests with 200 OK and 404 Not Found statuses). Hosted on GitHub.
- **Caido:** a web **security auditing toolkit** that security professionals use to intercept and view HTTP requests in real time while browsing. Website `https://caido.io`.
- **Some more tools: bettercap** (`https://www.bettercap.org`), **OWASP ZAP** (`https://www.zaproxy.org`), **WebSploit Framework** (`https://sourceforge.net`), **sslstrip** (`https://pypi.org`), and **Burp Suite** (`https://portswigger.net`).

Analogy: A mail room where every envelope is opened, copied, and resealed before delivery. Each tool is a different style of machine used in that mail room.

How it works (numbered steps, generic MITM proxy workflow):

1. Set the browser (or victim device) to send HTTP/HTTPS traffic through the tool's proxy port.
2. The tool shows each request and response in a log.
3. The tester searches the log for `Cookie:`, `Set-Cookie:`, and tokens in URLs.

4. The tester edits or replays a request to see if another user's session or a changed value is accepted.
5. The tester records the finding and the fix.

Real-world use: During a web app test, a tester runs Hetty or Caido as a proxy, logs in, and searches the log for the session cookie. They then replay a request with an old cookie after logout. The server still accepts it, which proves poor session expiry.

Tools and commands used here: Section 4 gives a one-row summary per tool. Command lines for bettercap, sslstrip, and the sniffers are in section 5.

AI angle: None shown. ADDED CONTEXT: Burp Suite has AI-assisted features in recent versions (verify); testers also paste suspicious requests into AI assistants for explanation.

Countermeasures and defense: The tools are legitimate security testing tools. The defense is to design the app so they cannot succeed: HTTPS with HSTS, short-lived random tokens, `HttpOnly` and `Secure` cookies (section 3.31), and monitoring.

EXAM FOCUS:

- Keyword "machine-in-the-middle HTTP proxy using logs and advanced search" -> Hetty.
- Keyword "web security auditing toolkit", "intercept and view HTTP requests in real time" -> Caido.
- The rest of the list: bettercap, OWASP ZAP, WebSploit Framework, sslstrip, Burp Suite.
- Trap: sslstrip downgrades HTTPS links to HTTP in a MITM position. It does not break TLS (ADDED CONTEXT).
- Trap: Hetty and Caido are proxies; Wireshark (section 3.26) is a sniffer. A proxy edits and replays requests. A sniffer only listens.

Memory trick: "H-C-B-O-W-S-B": Hetty, Caido, Bettercap, OWASP ZAP, WebSploit, Sslstrip, Burp.

3.25 Protecting against Session Hijacking [page 12]

What it is (simple words): A 14-point checklist of countermeasures. Each point is explained below.

1. **Use Secure Shell (SSH) or OpenSSH to create a secure communication channel.** SSH encrypts remote login and file transfer, so a sniffer sees only scrambled data. It replaces Telnet and rlogin.
2. **Implement log-out functionality for the user to end the session.** A logout button must destroy the session on the server, not just delete the cookie in the browser.

3. **Generate a session ID after a successful login and accept only session IDs generated by the server.** This stops session fixation and donation.
4. **Ensure that data in transit is encrypted and implement the defense-in-depth mechanism.** Use HTTPS or a VPN. Defense in depth means several layers so one failure does not give access.
5. **Use a string or long random numbers as session keys.** Long and random defeats prediction and brute force (section 3.7).
6. **Switch from a hub network to a switch network to reduce the risk of session hijacking attacks.** A hub sends every packet to every port. A switch sends a packet only to its destination port.
7. **Implement `timeout()` to destroy sessions when expired.** The server should delete a session after idle time or a maximum lifetime.
8. **Avoid including the session ID in the URL or query string.** URLs are logged, bookmarked, and shared.
9. **Ensure that client-side and server-side protection software are in the active state and up to date.** Antivirus and endpoint tools stop the Trojans behind Man-in-the-Browser attacks.
10. **Use strong authentication (such as Kerberos) or peer-to-peer virtual private networks (VPNs). Kerberos** is a ticket-based authentication protocol used in Windows domains. A VPN (virtual private network) encrypts a tunnel between two points.
11. **Configure appropriate internal and external spoof rules on gateways.** Drop packets arriving on the outside interface that claim an inside source address (and the reverse). This is ingress and egress filtering.
12. **Use IDS products or ARPwatch for monitoring ARP cache poisoning.** An IDS (intrusion detection system) raises alerts on suspicious traffic. **ARPwatch** watches IP-to-MAC pairings and reports changes.
13. **Enable browsers to verify website authenticity using network notary servers.** A **network notary** server lets many independent observers vouch that they see the same certificate for a site, which reveals a MITM that presents a different certificate to one victim.
14. **Use SFTP, AS2 managed file transfer, or FTPS to send data using encryption and digital certificates. SFTP** is file transfer over SSH. **FTPS** is FTP over TLS. **AS2** (Applicability Statement 2) is a protocol for secure business document exchange over HTTP/S.

Analogy: A bank vault that uses many controls at once: a guard (strong authentication), a sealed armored truck (encryption), a time lock (timeout), and a camera (IDS). Removing one control does not open the vault.

How the 14 points map to attacks:

Point(s)	Main attack they block
1, 4, 10, 14	Sniffing, MITM, replay (encryption)

Point(s)	Main attack they block
2, 7	Replay and long-lived stolen tokens
3, 5, 8	Prediction, fixation, donation
6, 11, 12	Network-level sniffing, ARP poisoning, spoofing
9	Man-in-the-Browser (malware)
13	MITM with a fake certificate

Real-world use: A company audits its web portal and finds the session ID in the URL and no logout invalidation. The fix list matches points 2, 3, 7 and 8, and the audit passes on retest.

Tools and commands used here: SSH/OpenSSH, ARPwatch, IDS, VPN software (section 4).

AI angle: ADDED CONTEXT: an AI assistant can turn this list into an application security checklist. Example prompt: "Convert these 14 session hijacking countermeasures into a test case table for a Java Spring web app, with a pass or fail column."

EXAM FOCUS:

- Must-memorize: the list has 14 items. Expect "which of these is a countermeasure" questions and "which is NOT" questions.
- Keywords by item: "log-out", "timeout()", "string or long random numbers", "hub to switch", "ARPwatch", "network notary", "SFTP, AS2, FTPS".
- Trap: "use a hub" is wrong. Switching from hub to switch is the countermeasure.
- Trap: "include the session ID in the URL" is wrong.

Memory trick: Group into five buckets: **Encrypt** (1, 4, 10, 14), **Session hygiene** (2, 3, 5, 7, 8), **Network** (6, 11, 12), **Endpoint** (9), **Trust** (13).

3.26 Session Hijacking Detection Tools [page 12]

What it is (simple words): Tools that watch network or host activity and raise an alert when a session looks hijacked. The slide names five.

- **USM Anywhere** (Unified Security Management, from AT&T Cybersecurity, `https://cybersecurity.att.com`): delivers threat detection, incident response, and compliance management across cloud, on-premises, and other environments.
- **Wireshark** (`https://www.wireshark.org`): lets you capture and interactively browse the traffic running on a computer network.
- **Quantum Intrusion Prevention System (IPS)** from Check Point (`https://www.checkpoint.com`).
- **SolarWinds Security Event Manager** (`https://www.solarwinds.com`).
- **IBM Security Network Intrusion Prevention System** (`https://www.ibm.com`).

Analogy: A building with cameras (Wireshark, which you review by hand), a central control room (USM Anywhere, SolarWinds SEM, which collect and correlate alerts), and automatic door locks (IPS products, which also block).

How detection works (numbered steps):

1. The tool captures traffic or collects logs from servers, firewalls and applications.
2. It compares what it sees to rules and baselines.
3. It flags signs of hijacking: the same session ID used from two IP addresses, duplicate or out-of-range TCP sequence numbers, a burst of ACK packets, a forged RST, a changed ARP entry, or many failed token guesses.
4. It raises an alert. An IPS may also drop the packets.
5. An analyst checks the alert in a packet capture.

ADDED CONTEXT: IDS vs IPS vs SIEM.

- **IDS** (Intrusion Detection System): detects and alerts.
- **IPS** (Intrusion Prevention System): detects and blocks inline.
- **SIEM** (Security Information and Event Management): collects logs from many sources and correlates them. USM Anywhere and SolarWinds Security Event Manager work in this space (verify product classification).

Real-world use: An analyst sees an alert "session cookie used from two countries in five minutes". They open the Wireshark capture, filter on the cookie value, confirm two client IPs, and terminate the session.

Tools and commands used here: Wireshark display filters (section 5.2).

AI angle: ADDED CONTEXT: many modern security platforms use machine learning to build a "normal" baseline for each user and raise an alert on a sudden change (new device, new country, abnormal request rate). A defender can ask an AI assistant: "Write a SIEM query that finds one session ID used by more than one source IP within 10 minutes."

EXAM FOCUS:

- Keyword "capture and interactively browse traffic" -> Wireshark.
- Keyword "threat detection, incident response, compliance across cloud and on-premises" -> USM Anywhere.
- Product names to match with vendors: Check Point (Quantum IPS), IBM (Security Network IPS), SolarWinds (Security Event Manager), AT&T Cybersecurity (USM Anywhere).
- Trap: Wireshark detects only if a human reads the capture. An IPS acts automatically.

Memory trick: "U-W-Q-S-I": USM, Wireshark, Quantum, SolarWinds, IBM.

3.27 Approaches to Prevent Session Hijacking: HSTS and Token Binding [page 13]

What it is (simple words): The slide gives two modern web defenses.

- **HTTP Strict Transport Security (HSTS):** a web security policy that protects HTTPS websites against MITM attacks. It allows web servers to **force** web browsers to interact with the site over the secure HTTPS protocol only.
- **Token Binding:** when a user logs on to a web application, the server generates a cookie with a SID, called a **token**. Token binding protects client-server communications against session hijacking attacks by tying that token to the secure connection.

Analogy:

- HSTS: a shop sign that says "Always use the armored door. Never the side door". The browser remembers the sign.
- Token Binding: a hotel key card that only works in the one room lock it was coded for. A copy is useless anywhere else.

How HSTS works (3 steps from the slide):

1. The client sends an HTTP request.
2. The server answers with an HSTS header, for example `Strict-Transport-Security: max-age=31536000; includeSubDomains`.
3. From then on the browser sends only HTTPS requests to that site, even if the user types `http://` or a link uses HTTP.

```
Client --01 HTTP request-----------------> Web Server
Client <-02 HSTS header------------------- Web Server
Client --03 HTTPS request----------------> Web Server
```

How Token Binding works (4 steps from the slide):

1. The browser starts a TLS session and the two sides check that Token Binding is supported ("TLS session - token binding?").
2. The browser generates a key pair for this site.
3. Both sides determine the TLS keys.
4. The browser sends a signature over the TLS keys plus its public key. The server ties the cookie/token to that key. A copy of the cookie sent from another machine or connection fails, because the thief lacks the private key.

ADDED CONTEXT: HSTS has a first-visit gap: the very first request can still be plain HTTP. Browsers close this with a built-in **HSTS preload list** (verify). Token Binding is defined in RFC 8471, 8472 and 8473 (verify); browser support has been limited and removed in some major browsers (verify). Newer approaches such as Device Bound Session Credentials follow the same idea (verify).

Real-world use: A bank turns on HSTS with a one-year max-age. A tester using sslstrip on a hotel Wi-Fi can no longer downgrade the bank's login page from HTTPS to HTTP for returning visitors.

Tools and commands used here: The `Strict-Transport-Security` response header (section 5.5). Test with `curl -I` (ADDED CONTEXT).

AI angle: None shown.

Countermeasures and defense: These are countermeasures. HSTS blocks SSL stripping (sslstrip). Token Binding blocks replay of stolen cookies on another connection.

EXAM FOCUS:

- Keywords "web security policy", "force browsers to use HTTPS" -> HSTS.
- Keywords "binds token to TLS", "cookie with SID called a token" -> Token Binding.
- Trap: HSTS does not stop session fixation or XSS. It only forces HTTPS.
- Trap: Token Binding is not the same as HttpOnly. HttpOnly hides the cookie from scripts. Token Binding makes a stolen cookie unusable.

Memory trick: "HSTS = Https Stays on. Token Binding = Token is Bonded to the connection."

3.28 Approaches to Prevent MITM Attacks: DNS over HTTPS [page 13]

What it is (simple words): DNS over HTTPS (DoH) is an enhanced version of the DNS protocol used to prevent snooping on a user's web activities or DNS queries during the DNS lookup process. The web queries and traffic are sent through encrypted HTTPS via **port 443**. Conventional DNS uses **port 53** in plain text.

Analogy: Asking directions out loud in a crowded square (conventional DNS) versus whispering the question inside a locked phone booth (DoH). Eavesdroppers can hear the first but not the second.

How it works (from the slide diagram):

1. A **DoH client** opens a secure tunnel on port 443 to a **DoH resolver**. Queries and answers travel as encrypted HTTPS.
2. A normal **DNS client** sends conventional DNS on port 53 to a DNS resolver. These queries are visible.
3. An attacker on the path can read or forge the port 53 traffic (the attacker symbol sits on the port 53 path).
4. The attacker cannot read or change the HTTPS tunnel (the slide shows a red X on that path).

```
DoH client --- port 443 secure tunnel ------------> DoH Resolver
DNS client --- port 53 conventional queries ------> DNS Resolver
                    ^
                    |
                 Attacker can read and forge port 53 only
```

ADDED CONTEXT: A related option is **DNS over TLS (DoT)**, which uses TCP port 853 (verify). DoH hides DNS inside normal web traffic. DoH protects the path to the resolver. It does not by itself prove that the answer data is correct. That job belongs to DNSSEC.

Real-world use: A traveler turns on DoH in the browser. On a hotel network, a rogue DNS server can no longer redirect the traveler's lookups for the bank to a fake site.

Tools and commands used here: Browser DoH setting; `curl` with a DoH URL (ADDED CONTEXT). See section 5.5.

AI angle: None shown.

Countermeasures and defense: DoH is the countermeasure. Pair with DNSSEC and HTTPS.

EXAM FOCUS:

- Keywords "DNS over HTTPS", "port 443", "prevent snooping of DNS queries" -> DoH.
- Port pair to memorize: DoH 443, conventional DNS 53.
- Trap: DoH hides queries from network eavesdroppers. It does not stop malware on the host, and it does not replace DNSSEC.
- The slide places it under "Approaches to Prevent MITM Attacks".

Memory trick: "DoH! 443": hide your DNS in the HTTPS crowd.

3.29 IPsec [page 14]

What it is (simple words): IPsec (Internet Protocol Security) is a protocol suite developed by the **IETF** (Internet Engineering Task Force) for securing IP communications by **authenticating and encrypting each IP packet** of a communication session. It is deployed widely to implement **VPNs** and for **remote user access** through dial-up connections to private networks.

Analogy: A courier service that seals every parcel (encryption) and stamps a tamper-proof label with the sender's identity (authentication). This works for any content inside, whatever the app.

The two security services (from the slide):

- **Authentication Header (AH):** provides data **authentication** of the sender. It proves who sent the packet and that it was not changed. It does not encrypt.

- **Encapsulating Security Payload (ESP):** provides both data **authentication and encryption (confidentiality)** of the sender. (The slide writes "Encapsulation Security Payload".)

Benefits of IPsec (5, from the slide):

1. Network-level peer authentication.
2. Data origin authentication.
3. Data integrity.
4. Data confidentiality (encryption).
5. Replay protection.

How it works (numbered steps, ADDED CONTEXT for the detail):

1. Two peers agree on policy and keys using **IKE** (Internet Key Exchange, UDP port 500; UDP 4500 for NAT traversal; verify).
2. The result is a **Security Association (SA)**: a one-way agreement with an algorithm, key and ID number (**SPI**, Security Parameter Index).
3. Each IP packet is wrapped with AH (IP protocol 51) or ESP (IP protocol 50) (verify numbers).
4. The receiver checks the authentication data, decrypts if ESP, and rejects replayed packets using a sequence counter.

Real-world use: A company links its London and New York offices with a site-to-site IPsec VPN. Staff on both LANs use plain internal apps, and everything between the two gateways is encrypted. A sniffer on the Internet path sees only ESP packets.

Why IPsec stops session hijacking: Each packet carries an authentication value. An attacker who injects, changes, replays, or resets a packet cannot produce a valid value without the key, so the receiver drops it. This covers TCP/IP hijacking, RST injection, and sniffing.

Tools and commands used here: OS IPsec stacks, strongSwan, VPN gateways (section 4). Check status with `ipsec statusall` on strongSwan (ADDED CONTEXT, verify).

AI angle: None shown.

EXAM FOCUS:

- Keywords "authenticating and encrypting each IP packet" -> IPsec.
- AH = authentication only. ESP = authentication + encryption.
- Trap: AH does NOT provide confidentiality. If the question needs "encrypt", the answer is ESP.
- Trap: IPsec works at the Network layer (Layer 3). SSL/TLS works above it.

Memory trick: "AH, I can see it but it is signed. ESP, Encrypted Secret Payload." AH = Authentic Header, ESP = Everything Secret Packed.

3.30 IPsec (Cont'd): Modes and Architecture [page 14]

What it is (simple words): IPsec runs in two **modes** and has an architecture made of protocols, algorithms, a DOI, policy, and key management.

Mode 1: Transport mode. Protects the data (payload) of the packet. The original IP header stays in place so routers can still forward it. Typically used host to host.

```
Transport-mode packet
+---------+--------+------------------+--------------+
| IP      | IPsec  | Transport data   | IPsec trailer|
| header  | header | (TCP UDP etc)    | (ESP only)   |
+---------+--------+------------------+--------------+
                     |<-- encrypted -->|
             |<------ authenticated -------->|
```

Mode 2: Tunnel mode. Wraps the **whole original IP packet** inside a new packet with a new outer IP header. Typically used gateway to gateway (site-to-site VPN). The slide shows Network 1 (Host 1) behind Gateway 1, the Internet, and Gateway 2 in front of Network 2 (Host 2).

```
Tunnel-mode packet
+--------+--------+--------+-----------+--------------+
| Outer  | IPsec  | Inner  | IP        | IPsec trailer|
| IP hdr | header | IP hdr | payload   | (ESP only)   |
+--------+--------+--------+-----------+--------------+
                   |<-------- encrypted -->|
         |<----------- authenticated ---->|
```

> **ADDED CONTEXT:** The exact span of the arrows differs slightly between AH and ESP. With ESP, the encrypted part covers the data and the trailer. With AH, nothing is encrypted and the authenticated part also covers most of the IP header. The slide shows ESP-style arrows. Verify.

IPsec architecture (from the slide diagram):

- **IPsec Architecture** is made of the **AH protocol** and the **ESP protocol**.
- Both use an **Authentication Algorithm** (to prove integrity and origin).
- ESP also uses an **Encryption Algorithm**.
- The algorithms plug into the **IPsec Domain of Interpretation (DOI)**, which defines how the pieces fit (the shared names, numbers and formats that all IPsec parts agree on; verify wording).
- **Policy** decides which traffic is protected and how.
- **Key Management** creates and distributes keys (IKE, ADDED CONTEXT).
- Policy and Key Management also connect back to the AH and ESP protocols in the diagram.

Analogy: Transport mode is putting a locked box inside your own labeled envelope: the address on the outside is yours, the content is locked. Tunnel mode is putting the

whole envelope inside a bigger envelope with a new address on the outside. The inner address is hidden.

Real-world use: A traveling employee's laptop uses transport mode to secure its connection to one server. A branch router uses tunnel mode so every computer in the branch is protected without any change.

Tools and commands used here: VPN clients and gateways. No command on the slide.

EXAM FOCUS:

- Transport mode: original IP header kept, end-to-end between hosts.
- Tunnel mode: original packet fully wrapped, new outer IP header, gateway-to-gateway. Hides the real source and destination.
- Pair rules: AH = no encryption in either mode. ESP = encrypts in either.
- Trap: tunnel mode is the one that adds an "outer IP header" and "inner IP header". The slide labels both clearly.
- Architecture parts to memorize: AH, ESP, authentication algorithm, encryption algorithm, DOI, policy, key management.

Memory trick: "Transport = Two hosts, header Talks. Tunnel = a Truck carries the whole packet."

3.31 Cookie Flags and Server-Side Session Hygiene (ADDED CONTEXT) [no slide]

> **ADDED CONTEXT:** Your screenshots do not show cookie flags, but the exam and every real defense rely on them. Treat this section as a required extension.

What it is (simple words): Small cookie attributes tell the browser how to protect a session cookie.

Attribute	What it does	Stops
`Secure`	Send the cookie only over HTTPS	Sniffing on HTTP
`HttpOnly`	Hide the cookie from JavaScript	XSS cookie theft
`SameSite=Strict/Lax`	Limit cookies on cross-site requests	CSRF
`Max-Age` / `Expires`	Set a lifetime	Long-lived stolen cookies
`Path` / `Domain`	Limit which URLs get the cookie	Wide exposure

Example header (read it left to right):

```
Set-Cookie: SID=9f2c...e1; Secure; HttpOnly; SameSite=Lax;
  Path=/; Max-Age=900
```

Related attacks to know (ADDED CONTEXT):

- **XSS (cross-site scripting):** a script injected into a page reads `document.cookie` and sends it to the attacker. `HttpOnly` blocks this.
- **CSRF (cross-site request forgery):** a malicious page makes the victim's browser send an authenticated request. It rides on the session without stealing it. `SameSite` and anti-CSRF tokens block it.
- **Session sidejacking:** sniffing a session cookie on an open network (public Wi-Fi) and reusing it. `Secure` plus HTTPS blocks it.
- **Session puzzling (session variable overloading):** an app reuses one session variable for different purposes, so the attacker reaches a state they should not (verify wording).

Server-side best practices:

1. Regenerate the SID at login and at privilege changes.
2. Use an idle timeout (for example 15 minutes) and an absolute lifetime.
3. Invalidate the session on the server at logout.
4. Ask for the password again before sensitive actions.
5. Never log full session IDs.
6. If using JWTs (JSON Web Tokens), keep lifetimes short and plan for revocation, because the server does not hold the session state.

Real-world use: A tester finds that a session cookie lacks `HttpOnly`. A stored XSS bug then lets them steal an admin cookie. The report lists both flaws and the fix: add `HttpOnly`, `Secure`, `SameSite`.

EXAM FOCUS:

- Keyword "script cannot read the cookie" -> `HttpOnly`.
- Keyword "only over HTTPS" -> `Secure`.
- Keyword "cross-site request" -> `SameSite` / CSRF token.
- Trap: `HttpOnly` does not stop sniffing. `Secure` does not stop XSS.

Memory trick: "S-H-S": Secure, HttpOnly, SameSite.

3.32 AI and Session Hijacking (ADDED CONTEXT) [no slide]

> **ADDED CONTEXT:** The CEH v13 AI exam adds AI topics to every module. Your Module 11 screenshots show none, so this section fills the gap. Verify details against current EC-Council material.

AI helps attackers by:

1. Writing fluent phishing emails that carry fixation, donation, or proxy links (sections 3.11, 3.12, 3.15). Tools of the "WormGPT" and "FraudGPT" style are marketed for this (verify).
2. Sorting huge piles of cookies and logs stolen by info-stealer malware and picking the valuable sessions.
3. Spotting patterns in captured tokens (section 3.7).
4. Cloning a voice or face (deepfake) to talk a help desk into resetting an account after a session is taken over.

AI helps defenders by:

1. Building a baseline for each user and flagging odd session behavior (new device, impossible travel, odd request rate).
2. Reviewing session code for weak random functions and missing flags.
3. Explaining packet captures and drafting detection queries.

Attacks on AI systems that involve sessions (OWASP Top 10 for LLM applications, verify the current list and numbers):

- **Prompt injection (LLM01):** hidden text in a web page or file makes an AI assistant leak data, for example by rendering a link or image whose URL contains the user's session data.
- **Sensitive information disclosure:** chat sessions leaking to the wrong user because of a bug. A widely reported case in March 2023 showed some users other people's chat titles (verify).
- **Excessive agency:** an AI agent that holds a user's login token and can act on their behalf. If its token is stolen, the attacker is the agent.
- **Stolen API keys or tokens:** attackers who steal cloud or AI service credentials resell or abuse them (sometimes called LLMjacking, verify).

Example prompts:

- Defender: "Review this login code. Does it regenerate the session ID after authentication? List every place the SID is created."
- Tester (authorized): "Explain what this Set-Cookie line means and which security flags are missing."

EXAM FOCUS:

- AI makes the lure (phishing, deepfake) better. The underlying attack (fixation, replay, MITM) is the same.
- Defense for AI agents is the same as for users: short-lived scoped tokens, least privilege, token binding, monitoring.
- Trap: do not answer "AI cannot hijack sessions". AI speeds up the steps. The session mechanisms are still the weakness.

Memory trick: "AI is the megaphone, not the lock pick." It makes the lure louder and the triage faster. The session flaw underneath is the same one this whole module teaches you to fix.

4. Tool Encyclopedia

Tool	What it does (simple)	Type/platform	Phase used	Exam keyword
Hetty	HTTP MITM proxy with logs and search	Web, open-source	Gaining access	"MITM HTTP proxy, logs"
Caido	Web security auditing, live HTTP intercept	Web, GUI	Gaining access	"intercept HTTP in real time"
bettercap	Swiss-army MITM: ARP spoof, sniff, proxy	Network, CLI/Web UI	Gaining access	"all-in-one MITM framework"
OWASP ZAP	Free web app scanner and intercept proxy	Web, open-source	Vulnerability/ Gaining access	"free, OWASP, web app proxy"
WebSploit Framework	Modular exploitation framework	Multi-module, CLI	Gaining access	"modular hijack/ exploit framework"
sslstrip	Downgrades HTTPS links to HTTP in a MITM path	Network, CLI	Gaining access	"strip SSL, force HTTP"
Burp Suite	Industry-standard web proxy, Repeater, Intruder	Web, GUI (Pro/ Free)	Gaining access	"Repeater, Intruder, Sequencer"
Wireshark	Captures and reads packets interactively	Network, GUI	Any (mainly detection)	"capture and browse traffic"
tcpdump	Command-line packet capture and TCP/IP analysis	Network, CLI	Monitor/Detection	"CLI, reads SEQ/ ACK"
Colasoft Packet Builder	Crafts custom packets (Windows GUI)	Network, GUI	Command injection	"craft packets, GUI, Windows"

Tool	What it does (simple)	Type/platform	Phase used	Exam keyword
hping3 (ADDED CONTEXT)	CLI packet crafting, spoofing, scanning	Network, CLI	Command injection	"craft TCP/UDP/ICMP packets"
Scapy (ADDED CONTEXT)	Python packet crafting and sniffing library	Network, scripting	Any	"script packets in Python"
Ettercap (ADDED CONTEXT)	Classic ARP spoofing and MITM suite	Network, CLI/GUI	Gaining access	"ARP poison, MITM suite"
ARPwatch (ADDED CONTEXT)	Watches IP-to-MAC pairs, alerts on change	Network, CLI daemon	Detection/ Defense	"monitor ARP cache poisoning"
USM Anywhere	Threat detection, IR, compliance (SIEM-like)	Cloud/on-prem	Detection	"cloud + on-prem threat mgmt"
Quantum IPS (Check Point)	Inline intrusion prevention appliance	Network appliance	Detection/ Prevention	"Check Point IPS"
SolarWinds Security Event Manager	Log collection and correlation (SIEM)	Software, on-prem	Detection	"SolarWinds, security events"
IBM Security Network IPS	Inline intrusion prevention appliance	Network appliance	Detection/ Prevention	"IBM IPS"
Evilginx / Modlishka (ADDED CONTEXT)	Reverse-proxy phishing that steals live sessions	Web, CLI	Gaining access	"phishing proxy steals MFA session"
Impacket ntlmrelayx (ADDED CONTEXT)	Relays NTLM auth captured via coercion (PetitPotam)	Network, CLI (Python)	Gaining access	"NTLM relay toolkit"
Certipy (ADDED CONTEXT)	Abuses AD CS to request certificates	AD, CLI (Python)	Gaining access/ Privilege escalation	"AD CS certificate abuse"

5. Command Sheet (I have no lab, so teach me to read commands)

> **ADDED CONTEXT:** Your screenshots name tools (Colasoft Packet Builder, tcpdump, Burp Suite, bettercap, sslstrip) but show no literal command lines. The commands below are standard usage for those named tools, added so you can read and recognize them on the exam. Verify exact flags against your installed tool version before running anything, and only test systems you are authorized to test.

5.1 tcpdump (named on the RST hijacking slide)

Purpose: Capture and read TCP/IP traffic from the command line, to check sequence numbers, flags, and addresses during a suspected hijack.

Full syntax:

```
tcpdump [-i interface] [-n] [-nn] [-X] [-v]
  [expression]
```

Flag table:

Flag	Meaning	When to use	Example
`-i`	Choose the capture interface	Always, unless default is right	`-i eth0`
`-n`	Do not resolve IP to names	Faster, avoids DNS noise	`-n`
`-nn`	Do not resolve IPs or ports	See raw port numbers	`-nn`
`-X`	Show packet bytes in hex and ASCII	Read payload content	`-X`
`-v`	Verbose: show TTL, flags, options	Spot source routing/ TTL games	`-v`

Demo commands (realistic scenario: watch a Telnet session for a hijack):

```
sudo tcpdump -i eth0 -nn -v tcp port 23
sudo tcpdump -i eth0 -nn 'tcp[13] & 4 != 0'
```

The second command uses a filter on byte 13 of the TCP header (the flags byte) to show only packets with the RST flag set (value 4).

Sample output and how to read it:

```
10:15:01.112 IP 10.0.0.5.23 > 10.0.0.9.51342:
  Flags [P.], seq 1201:1329, ack 1501, win 512
10:15:01.340 IP 10.0.0.9.51342 > 10.0.0.5.23:
  Flags [R], seq 1420, ack 0, win 0
```

Read it left to right: the first line is a normal data packet (`seq 1201:1329` means 128 bytes of data from sequence 1201 up to 1329) with the PSH flag `[P.]`. The second line shows a reset `[R]` at sequence 1420 with no acknowledgment, matching the RST hijacking pattern from section 3.19.

Common mistakes: forgetting `sudo`/root privileges (capture needs raw socket access); filtering on the wrong port; confusing `-n` (no name lookup) with `-nn` (no name or port lookup).

What the exam asks about it: recognizing tcpdump as a TCP/IP analysis tool paired with RST hijacking and TCP/IP hijacking; reading a short capture line to identify SEQ, ACK, and flags.

5.2 hping3 (ADDED CONTEXT, packet crafting for hijack labs)

Purpose: Craft and send custom TCP, UDP, or ICMP packets, including spoofed source addresses and forged flags such as RST, for authorized lab testing of hijacking and spoofing defenses.

Full syntax:

```
hping3 [-a spoofed_src] [-S|-R|-A] [-p port]
  [-s src_port] -c count target
```

Flag table:

Flag	Meaning	When to use	Example
`-a`	Spoof the source IP address	Simulate IP spoofing (3.18)	`-a 192.168.0.200`
`-S`	Set the SYN flag	Open a new TCP handshake	`-S`
`-R`	Set the RST flag	Simulate RST hijacking (3.19)	`-R`
`-p`	Destination port	Choose the service to target	`-p 23`
`-c`	Number of packets to send	Limit a lab test	`-c 1`

Demo commands (realistic lab scenario: send one spoofed RST):

```
sudo hping3 -a 192.168.0.200 -R -p 23 \
  -s 23 -c 1 192.168.0.100
```

This sends one RST packet to 192.168.0.100 port 23, pretending to come from 192.168.0.200 (the "server"), matching the RST hijacking diagram in section 3.19.

Sample output and how to read it:

```
HPING 192.168.0.100 (eth0 192.168.0.100): R set,
  40 headers + 0 data bytes
len=40 ip=192.168.0.100 ttl=64 DF id=0 sport=23
  flags=RA seq=0 win=0 rtt=1.0 ms
```

Read it as: the target answered with `flags=RA` (RST+ACK), which means the forged reset was accepted and the target believes the real server reset the connection.

Common mistakes: forgetting that spoofed packets need root privileges and a network that does not block spoofed source addresses (many networks apply egress filtering, section 3.18); sending many packets by accident without `-c`, which floods the target.

What the exam asks about it: recognizing hping3-style tools as how an attacker "crafts" the packets described in TCP/IP hijacking, RST hijacking, and IP spoofing sections; matching the `-a` flag to IP spoofing and `-R` to RST hijacking.

5.3 Ettercap / arpspoof (ADDED CONTEXT, for the ARP spoofing MITM slide)

Purpose: Poison ARP caches to route a victim's traffic through the attacker's host, as shown in section 3.22.

Full syntax (Ettercap):

```
ettercap -T -q -i interface -M arp:remote
  /victim_ip// /gateway_ip//
```

Flag table:

Flag	Meaning	When to use	Example
`-T`	Run in text mode (no GUI)	Scripted or terminal use	`-T`
`-q`	Quiet, less packet dump noise	Reduce clutter on screen	`-q`
`-i`	Choose the interface	Always needed	`-i eth0`

Flag	Meaning	When to use	Example
`-M arp:remote`	Run the ARP MITM module, both directions	Classic MITM, as in 3.22	`-M arp:remote`

Demo commands (realistic lab scenario: MITM a victim and the gateway):

```
sudo ettercap -T -q -i eth0 -M arp:remote \
  /192.168.0.100// /192.168.0.1//
```

This poisons 192.168.0.100 (victim) and 192.168.0.1 (gateway) so traffic between them flows through the attacker, matching the diagram in section 3.22.

Sample output and how to read it:

```
ARP poisoning victims:
 GROUP 1 : 192.168.0.100 AA:BB:CC:00:11:22
 GROUP 2 : 192.168.0.1   AA:BB:CC:00:11:22
Starting Unified sniffing...
```

Both "groups" now show the attacker's MAC address next to the real IP addresses, which confirms the poison succeeded and the attacker's host is now the man in the middle.

Common mistakes: forgetting to enable IP forwarding on the attacker's machine (the victim loses connectivity instead of being quietly relayed); running the attack across a VLAN boundary, which fails, because ARP spoofing only works on the same local segment.

What the exam asks about it: matching ARP spoofing tools to the "packet sniffer as an interface between client and server" description in section 3.22; knowing that the attacker must be on the same local network as the victim.

5.4 Wireshark session-hijacking display filters (ADDED CONTEXT)

Purpose: Narrow a large capture down to the packets that matter for a suspected hijack: resets, duplicate sequence numbers, or ARP replies.

Full syntax: typed directly into the Wireshark filter bar, same syntax style as Module 08's display filters.

Filter table:

Filter	Meaning	When to use	Example
`tcp.flags.reset==1`	Shows only RST packets	Hunt for RST hijacking	`tcp.flags.reset==1`

Filter	Meaning	When to use	Example
`tcp.analysis.duplicate_ack`	Wireshark's own duplicate-ACK flag	Spot desync/ACK storms	same
`tcp.analysis.retransmission`	Flags retransmitted segments	Spot desynchronization	same
`arp.duplicate-address-detected`	Flags conflicting ARP replies	Spot ARP spoofing	same
`ip.addr==10.0.0.5 && tcp.port==23`	One host's Telnet traffic	Isolate one suspect session	same

Demo commands (typed in the filter bar, realistic scenario: confirm an RST hijack on a Telnet session):

```
tcp.port==23 && tcp.flags.reset==1
ip.addr==10.0.0.9 && tcp.analysis.duplicate_ack
```

Sample output and how to read it: Wireshark's packet list shows a black-on-red row for an RST packet matched by the first filter, with the `Flags` column reading `0x0004 (RST)`. The second filter highlights rows Wireshark itself has already flagged in its "TCP analysis" column, meaning two different senders both answered with data for the same sequence number, a classic sign of a desynchronized, hijacked session.

Common mistakes: typing a capture filter (BPF syntax, used only when starting a capture) where a display filter is expected; forgetting `&&` between two conditions, which Wireshark rejects.

What the exam asks about it: reading a described filter and saying what it isolates; this mirrors the Module 08 Wireshark questions but applied to hijacking symptoms (RST, duplicate ACK) instead of sniffing.

5.5 SSH / OpenSSH and the Strict-Transport-Security header (ADDED CONTEXT)

Purpose: Show the two main defensive "commands" referenced by the countermeasure slides: starting an encrypted session with SSH (point 1, section 3.25) and checking a site's HSTS header (section 3.27).

Full syntax:

```
ssh [-p port] user@host
curl -I https://target_host
```

Flag table:

Flag	Meaning	When to use	Example
`-p` (ssh)	Non-default SSH port	Server uses a custom port	`-p 2222`
`-I` (curl)	Fetch headers only, no body	Quickly read HSTS/ Set-Cookie	`-I`

Demo commands (realistic scenario: confirm a site forces HTTPS and sets secure cookies):

```
ssh -p 22 admin@192.168.0.50
curl -I https://www.example.com
```

Sample output and how to read it:

```
HTTP/2 200
strict-transport-security: max-age=31536000;
  includeSubDomains
set-cookie: SID=9f2c...; Secure; HttpOnly;
  SameSite=Lax
```

Read it as: the `strict-transport-security` line confirms HSTS is active (section 3.27), and the `set-cookie` line confirms the three flags from section 3.31 are all present on this cookie.

Common mistakes: checking only `http://` instead of `https://`, which will not show the HSTS header the same way; assuming a missing header means HSTS is broken everywhere, when it may be set only on specific paths.

What the exam asks about it: matching "which command/tool confirms a security control is active" to a simple header check; recognizing SSH as the point-1 countermeasure for encrypting a session end to end.

6. Ports, Protocols, Standards and Acronyms

6.1 Ports and protocols seen in the screenshots and command sheet

Protocol/Port	Simple purpose	Default port/layer	Known weakness
TCP	Reliable, connection-based transport	Transport layer	Sequence numbers can be guessed
UDP	Fast, connectionless transport	Transport layer	No handshake, easy to forge replies
IP	Routes packets between networks	Network layer	Source address can be spoofed

Protocol/Port	Simple purpose	Default port/layer	Known weakness
ICMP	Network diagnostics and error messages	Network layer (no port)	Can be forged (redirects)
ARP	Resolves IP to MAC on a local segment	Data Link layer (no port)	No authentication, easy to spoof
Telnet	Remote terminal login	TCP 23	Cleartext, classic hijack target
HTTP	Web traffic	TCP 80	Cleartext session IDs can be sniffed
HTTPS	Encrypted web traffic	TCP 443	Still hijackable via MITB, fixation
SSH	Encrypted remote login	TCP 22	Strong when configured correctly
DNS	Resolves names to IPs	UDP/TCP 53	Spoofable without DNSSEC; UDP hijack target
DoH (DNS over HTTPS)	Encrypts DNS lookups inside HTTPS	TCP 443	Hides queries, does not prove data truth
DoT (DNS over TLS, ADDED)	Encrypts DNS lookups over TLS	TCP 853 (verify)	Same as DoH, different wrapper
SMB	Windows file/print sharing; PetitPotam channel	TCP 445	NTLM relay target (PetitPotam)
IKE (ADDED)	Negotiates IPsec keys and policy	UDP 500 (UDP 4500 NAT)	If weak, breaks the whole IPsec session
AH (IPsec)	Authenticates each IP packet	IP protocol 51 (verify)	No encryption by itself
ESP (IPsec)	Authenticates and encrypts each IP packet	IP protocol 50 (verify)	Misconfiguration can disable encryption

6.2 Standards and acronyms

Acronym	Full name	Why it matters here
SID	Session Identifier	The value every hijack technique tries to get or fake
ISN	Initial Sequence Number	Random starting point in a TCP handshake
SEQ	Sequence number	Tracks bytes sent; attackers must predict it

Acronym	Full name	Why it matters here
ACK	Acknowledgment number	Next byte expected; equals last SEQ + length
PRNG	Pseudo-Random Number Generator	Weak PRNGs make session tokens guessable
MITM	Man-in-the-Middle / Manipulator-in-the-Middle	Splits one connection into two, reads/edits it
MitB	Man-in-the-Browser / Manipulator-in-the-Browser	Trojan edits data inside the browser
DOM	Document Object Model	The live page structure a MitB Trojan reads/edits
CRIME	Compression Ratio Info-leak Made Easy	TLS compression leak attack (ADDED CONTEXT)
HSTS	HTTP Strict Transport Security	Forces browsers to use HTTPS only
DoH	DNS over HTTPS	Encrypts DNS lookups, prevents snooping
IPsec	Internet Protocol Security	Authenticates and encrypts IP packets
AH	Authentication Header	IPsec header for authentication only
ESP	Encapsulating Security Payload	IPsec header for authentication + encryption
DOI	Domain of Interpretation	Shared rules/format that IPsec pieces agree on
NTLM	NT LAN Manager	Older Windows auth protocol; relay target in PetitPotam
MS-EFSRPC	Microsoft Encrypting File System Remote Protocol	API abused by PetitPotam to coerce auth
AD CS	Active Directory Certificate Services	Issues certificates; relay target in PetitPotam
DC	Domain Controller	Holds Active Directory; coerced in PetitPotam
SMB	Server Message Block	Windows file-sharing protocol used as the relay channel
IETF	Internet Engineering Task Force	Body that develops IPsec and other Internet standards
VPN	Virtual Private Network	Encrypted tunnel; built using IPsec or TLS

Acronym	Full name	Why it matters here
IDS/IPS	Intrusion Detection/Prevention System	Detects (IDS) or blocks (IPS) hijack-like traffic
CSRF	Cross-Site Request Forgery	Rides on a session without stealing it (ADDED)
XSS	Cross-Site Scripting	Can steal a cookie that lacks HttpOnly (ADDED)

> **ADDED CONTEXT:** The module gives no port number for ARP because it works at the Data Link layer, below where ports exist. A question that asks "which port does ARP use" is testing whether you know ARP has no port at all, the same trap as in the Sniffing module.

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attack and its countermeasure

Attack	Countermeasure
Session token prediction (sequential/ timestamp/PRNG)	CSPRNG tokens, long length, rate limiting
MITM (network split into two connections)	HTTPS + HSTS, DAI, port security, VPN/IPsec
Man-in-the-Browser (Trojan browser extension)	Endpoint protection, up-to-date AV/OS, out-of-band checks
Session replay (capture and resend a token)	Short token lifetime, nonces, Token Binding
Session fixation (victim uses a known SID)	Regenerate SID at login, reject externally supplied SIDs
Proxy server hijacking (bogus link, relay)	URL/domain checks, phishing-resistant MFA (FIDO2)
CRIME (TLS/HTTP compression leak)	Disable TLS compression, guard HTTP compression
Forbidden attack (TLS nonce reuse)	Patch TLS stack, counter-based nonces, TLS 1.3
Session donation (attacker donates own SID)	Reject SIDs from URLs, re-authenticate for sensitive steps
TCP/IP hijacking (spoofed packets, SEQ guess)	Randomized ISNs, encryption (SSH/TLS/IPsec), switched LAN
IP spoofing: source routed packets	Disable IP source routing, ingress/egress filtering

Attack	Countermeasure
RST hijacking (forged reset)	TCP-AO/IPsec, RFC 5961 challenge ACK, encrypted sessions
Blind hijacking (inject with no visibility)	Encryption, randomized ISNs, avoid plaintext protocols
UDP hijacking (forged reply wins the race)	DNSSEC, random source ports/IDs, DoH/DoT, DTLS
MITM via forged ICMP/ARP spoofing	Dynamic ARP Inspection, static ARP, disable ICMP redirects
PetitPotam (coerced NTLM relay to AD CS)	Disable NTLM, enable EPA on AD CS, patch, SMB signing

7.2 Look-alike pairs

- **Spoofing vs Hijacking.** Difference: spoofing starts a brand-new session with stolen credentials; hijacking seizes an already-active session the victim created. Tell them apart: did the server see a fresh login (spoofing), or was an existing connection taken over (hijacking)?
- **MITM vs Man-in-the-Browser.** Difference: MITM sits on the network path between two systems; MitB sits inside the victim's own browser, after decryption. Tell them apart: does HTTPS stop the attack? If yes, it was network-level MITM; if HTTPS does not help, think MitB.
- **Session Replay vs Session Fixation.** Difference: replay reuses a token the attacker captured AFTER the victim logged in; fixation gives the victim a token the attacker already knew BEFORE the victim logs in. Tell them apart: did the attacker know the SID before or after the victim's login?
- **Session Fixation vs Session Donation.** Difference: in fixation, the victim ends up using the server's normal session, and the attacker later reuses the same SID to become the victim; in donation, the victim's actions are saved into the ATTACKER's own already-logged-in account. Tell them apart: whose account does the data end up in?
- **TCP/IP Hijacking vs RST Hijacking.** Difference: TCP/IP hijacking injects data to take over and continue the session; RST hijacking forges a reset to kill the session outright. Tell them apart: does the attacker end up WITH the session (TCP/IP hijacking) or does the session simply DIE (RST hijacking)?
- **Blind Hijacking vs UDP Hijacking.** Difference: blind hijacking is a TCP attack where the attacker cannot see the response; UDP hijacking is a UDP attack that is a race to answer first, and the attacker usually does see its own forged reply succeed. Tell them apart: is this TCP with hidden replies (blind), or UDP with a timing race (UDP hijacking)?
- **IP Spoofing (source routed) vs Blind Hijacking.** Difference: source routing lets a spoofing attacker SEE replies by routing them back through themselves; blind hijacking has source routing DISABLED, so the attacker cannot see

anything. Tell them apart: does the scenario mention the attacker reading responses (source routed) or acting with no visibility (blind)?

- **CRIME vs Forbidden Attack.** Difference: CRIME leaks secret data by watching compressed message size; the Forbidden attack forges content by reusing a cryptographic nonce. Tell them apart: does the question mention compression (CRIME) or nonce reuse (Forbidden attack)?
- **HSTS vs Token Binding.** Difference: HSTS forces the browser to use HTTPS at all; Token Binding ties a specific session token to the exact TLS connection it was issued on. Tell them apart: is the fix about which protocol is used (HSTS), or about making a stolen token useless elsewhere (Token Binding)?
- **AH vs ESP (IPsec).** Difference: AH authenticates only; ESP authenticates AND encrypts. Tell them apart: does the question need confidentiality/encryption? If yes, the answer is ESP, never AH alone.
- **Transport Mode vs Tunnel Mode (IPsec).** Difference: transport mode keeps the original IP header and protects host-to-host traffic; tunnel mode wraps the whole original packet inside a new one, for gateway-to-gateway VPNs. Tell them apart: is only the payload protected (transport), or is the entire original packet hidden inside a new outer packet (tunnel)?

8. Exam Cheat Sheet

8.1 Keyword to answer

Keyword in the question	Likely answer
"seizes control of an existing/active TCP session"	Session hijacking
"initiates a new session with stolen credentials"	Spoofing attack
"authentication only happens at the start of a session"	Why hijacking works
"splits into client-to-attacker, attacker-to-server legs"	MITM
"Trojan edits form data inside the browser via the DOM"	Man-in-the-Browser
"captures and resends an authentication token"	Session replay attack
"lures victim to authenticate with a known/fixed session ID"	Session fixation
"forwards requests on behalf of the victim, acts as a proxy"	Proxy server hijacking
"reused cryptographic nonce during TLS"	Forbidden attack
"attacker donates his/her own SID to the victim"	Session donation attack

Keyword in the question	Likely answer
"transport/Internet protocols, below the app layer"	Network-level hijacking
"HTTP session ID", "cookie-based takeover"	Application-level hijacking
"spoofed packets, predicts sequence number, same network"	TCP/IP hijacking
"trusted host's IP, source-routed, attacker destination"	IP spoofing: source routed packets
"forged authentic-looking RST packet, predicted ACK"	RST hijacking
"inject data/commands, cannot see the response"	Blind hijacking
"forged server reply arrives before the real one"	UDP hijacking
"changes default gateway, forged ICMP, ARP spoofing"	MITM via forged ICMP/ARP
"DC forced to authenticate, MS-EFSRPC, AD CS cert"	PetitPotam
"forces browsers to use HTTPS only"	HSTS
"binds a session token to the TLS connection"	Token Binding
"encrypts DNS lookups over port 443"	DNS over HTTPS (DoH)
"authenticates and encrypts every IP packet, used for VPNs"	IPsec
"authentication only, no encryption" (IPsec)	AH
"authentication and encryption" (IPsec)	ESP
"original IP header kept, host to host"	IPsec Transport mode
"whole packet wrapped, gateway to gateway"	IPsec Tunnel mode
"colasoft packet builder, tcpdump"	RST hijacking tools
"HTTP toolkit, MITM proxy with logs"	Hetty
"web security auditing, intercepts HTTP live"	Caido

8.2 Must-memorize lists

- Five-stage hijacking process, in order: Sniff, Monitor, Session Desynchronization, Session ID Prediction, Command Injection.
- Two levels of hijacking: Network-level and Application-level.
- Eight application-level techniques, in slide order: Predict token, MITM, Man-in-the-Browser, Session Replay, Session Fixation, Proxy Server, Forbidden Attack, Session Donation.

- Six network-level techniques: Blind hijacking, UDP hijacking, TCP/IP hijacking, RST hijacking, MITM (packet sniffer), IP spoofing (source routed packets).
- Three ways to deliver a fixed SID (session fixation): URL argument, hidden form field, cookie.
- Fourteen-point session hijacking countermeasure checklist (section 3.25).
- Five benefits of IPsec: peer authentication, data origin authentication, data integrity, data confidentiality, replay protection.
- Two IPsec headers: AH (authentication only), ESP (authentication plus encryption). Two IPsec modes: Transport, Tunnel.

8.3 Numbers and defaults

Item	Value
Telnet port	TCP 23
HTTP / HTTPS ports	TCP 80 / TCP 443
SSH port	TCP 22
DNS port (conventional)	UDP/TCP 53
DoH port	TCP 443
SMB port (PetitPotam channel)	TCP 445
ACK formula	Next ACK = previous SEQ + data length
Session hijacking countermeasure list length	14 points
Man-in-the-Browser attack steps	14 steps
Session Donation attack steps	5 steps
PetitPotam attack steps	5 steps

8.4 Rules for picking the BEST option

- If both "spoofing" and "hijacking" appear as options and the scenario shows a user ALREADY logged in before the attacker acts, pick hijacking; if the attacker supplies credentials to start a brand-new session, pick spoofing.
- If a question gives a specific named mechanism (nonce reuse, TLS compression, MS-EFSRPC) do not answer with a generic "MITM attack"; pick the specific named attack (Forbidden attack, CRIME, PetitPotam) because the exam rewards precise identification.
- If a defense question offers both "use HTTPS" and a more specific fix (HSTS, Token Binding, DNSSEC, IPsec ESP), prefer the specific, named fix that matches the exact flaw in the question, since "use HTTPS" alone is often a true-but-incomplete distractor.

- If asked to choose between AH and ESP and the question mentions "confidentiality" or "encryption" anywhere, the answer must be ESP; AH can never provide that benefit by itself.
- If a network-level attack's "best next step to confirm" is asked, prefer reading a packet capture (Wireshark/tcpdump) over guessing from symptoms alone, since the module repeatedly ties detection to actually looking at SEQ/ACK/RST values.

9. Practice Questions

These 30 questions are original. Each answer and explanation sits directly below its question.

Question 1. A victim logs into a web banking portal using their own username and password. Ten minutes later, while the victim is still browsing, an attacker on the same coffee shop Wi-Fi forges the victim's IP address and takes over the TCP connection using a predicted sequence number. What has the attacker done?

- A. Spoofing attack
- B. Session hijacking
- C. Phishing attack
- D. Session donation

Answer: B. The attacker seized control of an already-active session that the victim created, which is the definition of session hijacking.

- A is wrong: spoofing starts a brand-new session with stolen credentials; here the victim's own live session was taken over.
- C is wrong: no fake login page or credential theft is described.
- D is wrong: session donation requires the attacker to give the victim the attacker's own SID first; here the attacker took the victim's SID.

Question 2. Which statement correctly separates spoofing from hijacking?

- A. Spoofing seizes an existing session; hijacking creates a new one
- B. Spoofing creates a new session with stolen credentials; hijacking seizes an existing session
- C. Both attacks always require network-level packet sniffing
- D. Spoofing and hijacking are two names for the same attack

Answer: B. This is the exact distinction given on the Spoofing vs Hijacking slide.

- A is wrong: it reverses the correct definitions.
- C is wrong: spoofing can happen purely with stolen credentials and no sniffing at all.
- D is wrong: the module explicitly separates them as different attacks.

Question 3. Put the five stages of the session hijacking process in the correct order.

- A. Monitor, Sniff, Command Injection, Session Desync, Session ID Prediction
- B. Sniff, Monitor, Session Desynchronization, Session ID Prediction, Command Injection
- C. Session ID Prediction, Sniff, Monitor, Command Injection, Session Desync
- D. Command Injection, Monitor, Sniff, Session ID Prediction, Session Desync

Answer: B. This is the five-stage order shown bottom to top on the Session Hijacking Process slide.

- A is wrong: Monitor cannot happen before Sniff; you must see traffic before you can monitor it.
- C is wrong: Session ID Prediction happens near the end, not first.
- D is wrong: Command Injection is the last stage, not the first.

Question 4. A tester finds that a company's internal app issues session tokens like `A1001`, `A1002`, `A1003` to successive users. What weakness does this show, and what is the best fix?

- A. Weak PRNG seed; switch to a hub network
- B. Sequential tokens; use a cryptographically secure random generator
- C. Small token space; disable logout
- D. Timestamp-based tokens; add a time zone field

Answer: B. The tokens increase by a fixed step with a constant prefix, which is the textbook description of sequential tokens; the fix is a CSPRNG-generated, non-sequential token.

- A is wrong: nothing here shows a PRNG seed problem, and switching to a hub network has nothing to do with token generation.
- C is wrong: small token space is about the total count of possible values, not an increasing counter, and disabling logout would make things worse.
- D is wrong: there is no timestamp shown in the token, only a counter.

Question 5. An attacker splits one TCP connection into a "client-to-attacker" connection and an "attacker-to-server" connection, then reads and modifies the HTTP traffic in between. Which attack is this?

- A. Session donation
- B. Man-in-the-Browser
- C. Man-in-the-Middle (Manipulator-in-the-Middle)
- D. Session fixation

Answer: C. Splitting one connection into two separate legs through the attacker is the exact mechanism described for MITM.

- A is wrong: session donation does not split a TCP connection; it gives the victim the attacker's own SID.
- B is wrong: Man-in-the-Browser works inside the browser's own process, not by splitting a network connection in two.
- D is wrong: session fixation works by pre-setting a known SID, not by splitting a live connection.

Question 6. A victim's computer is infected with a Trojan that installs a malicious browser extension. The extension waits until the victim logs into their bank, then uses the DOM to silently change the payee account number on a submitted form. The bank's receipt, shown back to the victim, still displays the original payee. Which attack is this?

- A. Session replay
- B. Man-in-the-Browser (Manipulator-in-the-Browser)
- C. Session hijacking using proxy servers
- D. UDP hijacking

Answer: B. A Trojan-installed extension that reads and edits form data through the DOM, then hides the change from the user, is the Man-in-the-Browser attack described in the module.

- A is wrong: session replay resends a previously captured token; it does not edit live form data.
- C is wrong: proxy server hijacking relays a whole transaction through an attacker-controlled server reached via a bogus link, not a browser extension.
- D is wrong: UDP hijacking is a network-level race to answer a UDP request first; it has nothing to do with a browser extension.

Question 7. An attacker eavesdrops on a login exchange, captures the user's authentication token, and later resends that exact same token to the server to gain access. What is this attack, and what single fix would most directly stop it?

- A. Session fixation; regenerate the SID at login
- B. Session replay; add a nonce or short lifetime so an old token is rejected
- C. RST hijacking; disable source routing
- D. PetitPotam; disable NTLM

Answer: B. Capturing and resending an authentication token is the definition of session replay; a nonce (used once) or a short token lifetime defeats a captured, reused token.

- A is wrong: in fixation the attacker supplies the SID before login, not after capturing one.

- C is wrong: RST hijacking tears down a connection with a forged reset; it does not involve capturing and resending a token.
- D is wrong: PetitPotam is a Windows domain controller NTLM-relay attack, unrelated to a replayed authentication token.

Question 8. An attacker emails a victim a link that already contains a valid, server-issued session ID in the query string. The victim clicks the link and logs in, unaware that the attacker already knows that exact session ID. The attacker then uses the same SID to access the victim's authenticated session. What is this attack?

- A. Session donation
- B. Session fixation
- C. CRIME attack
- D. Blind hijacking

Answer: B. The attacker supplied (fixed) a known SID to the victim before the victim authenticated, then reused it after login; this is session fixation.

- A is wrong: in donation the SID belongs to the attacker's own logged-in account, and the victim's data ends up there, not the other way around.
- C is wrong: CRIME is a TLS/HTTP compression side-channel attack, not a pre-set session ID lure.
- D is wrong: blind hijacking is a network-level TCP injection attack with no response visibility, unrelated to emailing a link.

Question 9. An attacker already has a valid, logged-in session on a shopping site. The attacker emails that same session ID to a victim and lures the victim to click it. The victim enters their credit card details on the site, not knowing that the data is being saved to the attacker's account. What is this attack?

- A. Session fixation
- B. Session donation
- C. Proxy server hijacking
- D. Man-in-the-Middle

Answer: B. The attacker donated their own already-authenticated SID to the victim, so the victim's actions were saved into the attacker's account; this matches session donation exactly.

- A is wrong: in fixation the victim ends up in their own normal, server-issued session, which the attacker later reuses; here the victim is using the attacker's account the whole time.
- C is wrong: proxy hijacking relays traffic live through a bogus attacker server between the victim and the real site, with no mention of a shared SID.
- D is wrong: MITM splits a connection into two legs; this scenario describes sharing one pre-existing SID, not intercepting traffic.

Question 10. A security researcher demonstrates an attack on an HTTPS session by exploiting the reuse of a cryptographic nonce, allowing them to inject forged content and steal sensitive data the victim is tricked into entering. What is this attack called?

- A. CRIME attack
- B. Forbidden attack
- C. RST hijacking
- D. Session donation

Answer: B. Exploiting nonce reuse to hijack an HTTPS session and inject forged content is specifically named the Forbidden attack.

- A is wrong: CRIME exploits compression ratio changes, not nonce reuse.
- C is wrong: RST hijacking forges a TCP reset packet; it has nothing to do with TLS nonces.
- D is wrong: session donation shares a pre-existing SID; it does not involve TLS cryptography at all.

Question 11. Which TWO levels does the module split session hijacking into, based on the OSI model?

- A. Physical-level and Data Link-level
- B. Network-level and Application-level
- C. Transport-level and Presentation-level
- D. Session-level and Physical-level

Answer: B. The module explicitly defines Network-Level Hijacking (intercepting TCP/UDP packets) and Application-Level Hijacking (stealing the HTTP session ID).

- A is wrong: these are OSI layers, but not the two categories this module uses to classify hijacking.
- C is wrong: the module does not use these layer names as its two hijacking categories.
- D is wrong: "Session-level" is not one of the two categories the slide names (it is loosely related to OSI's Session layer, but the module uses "Application-level" instead).

Question 12. During a lab exercise, an attacker on the same LAN as a victim watches a TCP handshake, predicts the next sequence number, and injects data into the live session before the victim's next packet arrives, taking over the conversation with the server. Which network-level attack is this?

- A. TCP/IP hijacking
- B. UDP hijacking
- C. Blind hijacking
- D. PetitPotam

Answer: A. Using spoofed packets plus a predicted sequence number to seize control of a live TCP connection, while on the same network as the victim, is the textbook description of TCP/IP hijacking.

- B is wrong: UDP hijacking targets a connectionless UDP exchange, with no sequence numbers to predict, and no TCP handshake.
- C is wrong: blind hijacking specifically means the attacker cannot see any server responses; this scenario does not say that.
- D is wrong: PetitPotam is a Windows Active Directory NTLM-relay technique, not a generic LAN TCP takeover.

Question 13. In the TCP/IP hijacking sequence diagram, the server sends `DATA=91<CltSEQ 1329>` and the client's last acknowledgment was for sequence 1329. What is the correct next acknowledgment number the server should send back?

- A. 1329
- B. 1420
- C. 91
- D. 1440

Answer: B. ACK = previous SEQ + data length = 1329 + 91 = 1420.

- A is wrong: that repeats the starting sequence number instead of adding the data length.
- C is wrong: that is only the data length, not the new acknowledgment value.
- D is wrong: 1440 is the next step's ACK in the full diagram (1420 + 20), not the answer to this specific pair of numbers.

Question 14. An attacker spoofs a trusted host's IP address and uses IP source routing so that reply packets are routed back through the attacker rather than to the real trusted host. What is this technique called?

- A. Blind hijacking
- B. IP spoofing using source-routed packets
- C. UDP hijacking
- D. Session replay

Answer: B. Using IP source routing so the attacker both spoofs a trusted host's address AND sees the resulting replies is exactly the "IP Spoofing: Source Routed Packets" technique.

- A is wrong: blind hijacking specifically happens when the attacker has NO visibility of responses, often because source routing is disabled; this question describes the opposite case.
- C is wrong: UDP hijacking is a connectionless race to reply first, with no mention of source routing or a trusted host's IP.

- D is wrong: session replay resends a captured token; it does not involve IP source routing at all.

Question 15. An attacker sends a forged TCP packet with the RST flag set, a spoofed source address matching the real server, and an accurately predicted acknowledgment number. The victim's machine believes the real server sent it and closes the connection. What is this attack, and which tool is specifically named in the module for crafting such packets?

- A. Blind hijacking; Wireshark
- B. RST hijacking; Colasoft Packet Builder
- C. UDP hijacking; Caido
- D. PetitPotam; Ettercap

Answer: B. Forging an authentic-looking reset packet with a predicted ACK number is RST hijacking, and the module names Colasoft Packet Builder as a packet-crafting tool for it (alongside tcpdump for analysis).

- A is wrong: blind hijacking is about injecting data with no response visibility, not resetting a connection, and Wireshark only captures traffic, it does not craft packets.
- C is wrong: UDP hijacking targets UDP, not a TCP RST flag, and Caido is an HTTP proxy tool, not a packet crafter.
- D is wrong: PetitPotam targets Windows domain controllers over MS-EFSRPC, unrelated to a forged TCP RST packet.

Question 16. An attacker intercepts a victim's UDP request to a DNS server and sends back a forged reply before the real server's response arrives. The victim's resolver accepts the first answer it receives. Which network-level attack is this?

- A. TCP/IP hijacking
- B. RST hijacking
- C. UDP hijacking
- D. Session fixation

Answer: C. Winning the race to answer a victim's UDP request before the real server replies is the exact description of UDP hijacking given in the module.

- A is wrong: TCP/IP hijacking requires a TCP handshake and sequence number prediction; UDP has neither.
- B is wrong: RST hijacking forges a TCP reset packet to end a connection, not a race to answer first.
- D is wrong: session fixation is an application-level attack that presets a session ID; it has nothing to do with UDP timing races.

Question 17. An attacker changes a victim's default gateway using forged ARP replies, so the victim's and server's traffic is routed through the attacker's host, where

a packet sniffer intercepts it. Which two techniques does the module name for this type of attack?

- A. CRIME and BREACH
- B. Forged ICMP and ARP spoofing
- C. TCP/IP hijacking and RST hijacking
- D. Session fixation and session donation

Answer: B. The module names forged ICMP messages and ARP spoofing as the two techniques used to reroute packets through the hijacker's host in this MITM attack.

- A is wrong: CRIME and BREACH are compression-based TLS/HTTP leaks, unrelated to default gateway manipulation.
- C is wrong: those are both separate network-level attacks on TCP, not the two techniques named for this gateway-rerouting MITM attack.
- D is wrong: those are application-level session ID attacks, not network-level gateway rerouting techniques.

Question 18. In a PetitPotam attack, the attacker forces a domain controller to authenticate to the attacker's SMB server using which Microsoft API, and where is that NTLM authentication then relayed?

- A. MS-RPC; to the victim's own machine
- B. MS-EFSRPC; to the Active Directory Certificate Services (AD CS) server
- C. Kerberos; to the DNS server
- D. LDAP; to the file server

Answer: B. PetitPotam uses the Microsoft Encrypting File System Remote Protocol (MS-EFSRPC) API to coerce authentication, then relays the captured NTLM authentication to the AD CS server to generate a certificate with admin-level privileges.

- A is wrong: MS-RPC is a generic remote procedure call framework, not the specific API PetitPotam abuses, and relaying to the victim's own machine would not achieve anything.
- C is wrong: Kerberos is a different Windows authentication protocol, and DNS servers are not the relay target in this attack.
- D is wrong: LDAP is the directory access protocol, and a plain file server is not where the NTLM authentication is relayed in this attack.

Question 19. Which pair of tools does the module describe as "an HTTP toolkit for MITM proxy attacks using logs and advanced search" and "a web security auditing toolkit for real-time HTTP interception" , respectively?

- A. Burp Suite and OWASP ZAP
- B. bettercap and sslstrip
- C. Hetty and Caido

- D. Wireshark and tcpdump

Answer: C. The module describes Hetty exactly this way (MITM HTTP proxy with logs and advanced search) and Caido exactly this way (web security auditing, real-time HTTP interception).

- A is wrong: Burp Suite and OWASP ZAP are also session hijacking tools named on the same slide, but the specific descriptions quoted in the question match Hetty and Caido, not these two.
- B is wrong: bettercap is a broader MITM framework and sslstrip downgrades HTTPS to HTTP; neither matches the quoted descriptions.
- D is wrong: Wireshark and tcpdump are sniffing and analysis tools, not the MITM proxy/auditing tools described in the question.

Question 20. Which session hijacking countermeasure, from the 14-point list, most directly stops a victim's old session from being reused after the user is done, even if the browser's cookie is still present?

- A. Switch from a hub network to a switch network
- B. Implement log-out functionality and `timeout()` to destroy expired sessions
- C. Use SFTP, AS2, or FTPS for file transfer
- D. Use IDS products or ARPwatch for monitoring ARP cache poisoning

Answer: B. Logout and session timeout both actively destroy the session on the server, so a leftover cookie becomes useless, which directly stops reuse of an old session.

- A is wrong: switching to a switched network reduces the risk of sniffing on a LAN; it does nothing to expire an old session.
- C is wrong: those protocols protect file transfers in transit; they do not govern web session lifetime.
- D is wrong: ARP monitoring detects a specific LAN-based attack technique; it does not expire sessions.

Question 21. A web server sends the header `Strict-Transport- Security: max-age=31536000; includeSubDomains` to a browser. What is the direct effect of this header?

- A. It encrypts the session token so it cannot be read on the wire
- B. It forces the browser to use HTTPS only for this site and its subdomains for about one year
- C. It binds the session cookie to the current TLS connection
- D. It tells the browser to accept any certificate presented by the site

Answer: B. This is exactly what HSTS does: the browser remembers to use HTTPS only, for the stated `max-age` (one year, 31,536,000 seconds), and across subdomains because of `includeSubDomains`.

- A is wrong: HSTS does not encrypt anything itself; it only forces the use of an already-encrypting protocol (HTTPS).
- C is wrong: binding a token to a specific TLS connection describes Token Binding, a separate mechanism.
- D is wrong: HSTS does the opposite; it enforces strict HTTPS use and does not relax certificate checking.

Question 22. Which statement correctly distinguishes Token Binding from simply using an `HttpOnly` cookie flag?

- A. Token Binding stops JavaScript from reading the cookie; HttpOnly binds the cookie to the TLS connection
- B. Token Binding cryptographically ties a session token to the specific TLS connection it was issued on; HttpOnly hides the cookie from browser scripts
- C. They are two names for the exact same browser feature
- D. Token Binding only works over HTTP, never HTTPS

Answer: B. Token Binding makes a stolen token useless on a different connection by tying it to the TLS session; HttpOnly instead stops JavaScript (and so XSS attacks) from reading the cookie value at all.

- A is wrong: it swaps the two definitions.
- C is wrong: they protect against different threats (token theft via another connection versus theft via a malicious script) and are not the same feature.
- D is wrong: Token Binding is specifically a TLS-layer feature, so it requires HTTPS, not plain HTTP.

Question 23. A company enables DNS over HTTPS (DoH) for its laptops. What does this specifically protect against on an untrusted network such as a hotel Wi-Fi?

- A. It stops a Man-in-the-Browser Trojan from editing form data
- B. It prevents a local network eavesdropper from reading or forging the user's DNS lookups, by sending them encrypted over port 443
- C. It replaces the need for HTTPS entirely
- D. It guarantees that the DNS answer itself is cryptographically authentic

Answer: B. DoH wraps DNS queries inside an encrypted HTTPS tunnel on port 443, so a local attacker watching plain port 53 traffic can no longer read or tamper with the lookups.

- A is wrong: DoH is a DNS protection; it has no effect on a browser extension Trojan, which operates after DNS resolution is already done.

- C is wrong: DoH only protects DNS lookups; it does not remove the need to use HTTPS for the actual website traffic.
- D is wrong: proving the DNS answer's authenticity is the job of DNSSEC; DoH protects the transport, not the data's cryptographic origin.

Question 24. Which IPsec component provides BOTH data authentication and data confidentiality (encryption), unlike its counterpart which only authenticates?

- A. Authentication Header (AH)
- B. Encapsulating Security Payload (ESP)
- C. Internet Key Exchange (IKE)
- D. Domain of Interpretation (DOI)

Answer: B. ESP is explicitly described as providing both data authentication and encryption (confidentiality); AH provides authentication only.

- A is wrong: AH is the counterpart that provides authentication only, with no encryption at all.
- C is wrong: IKE negotiates the keys and policy used by AH/ESP; it is not itself one of the two security services described on the slide.
- D is wrong: the DOI defines the shared rules and formats IPsec components agree on; it does not itself authenticate or encrypt data.

Question 25. A company connects its two office gateways with a site-to-site VPN. The entire original IP packet from each internal host, including its original header, is wrapped inside a brand-new outer IP packet for the trip between the gateways. Which IPsec mode is this?

- A. Transport mode
- B. Tunnel mode
- C. Aggressive mode
- D. Quick mode

Answer: B. Wrapping the entire original packet, including its header, inside a new outer IP packet for gateway-to-gateway transport is exactly how IPsec Tunnel mode is described in the module.

- A is wrong: Transport mode keeps the original IP header exposed and protects only the payload, typically for host-to-host traffic, not a fully wrapped packet between gateways.
- C and D are wrong: "Aggressive mode" and "Quick mode" are IKE negotiation modes (ADDED CONTEXT, outside this module's two named IPsec modes), not the Transport/Tunnel distinction the slide teaches.

Question 26. An analyst runs `tcpdump -i eth0 -nn 'tcp[13] & 4 != 0'` on a suspicious host. What is this filter specifically designed to show?

- A. Every packet on port 23 regardless of flags

- B. Only packets with the TCP RST flag set, useful for spotting RST hijacking
- C. Only ARP replies, useful for spotting ARP spoofing
- D. Only UDP packets, useful for spotting UDP hijacking

Answer: B. Byte 13 of the TCP header holds the flags; a bitwise AND with 4 isolates the RST bit, so this filter shows only reset packets, which is exactly what an analyst would use to confirm RST hijacking.

- A is wrong: the filter does not reference port 23 at all, and it excludes any packet without the RST flag.
- C is wrong: ARP has no TCP header and no flags byte, so this filter cannot show ARP traffic.
- D is wrong: UDP packets have no TCP flags byte either, so this tcp[] based filter cannot match them.

Question 27. An attacker wants to test, in an authorized lab, whether a server accepts a forged TCP RST packet that appears to come from 192.168.0.200. Which hping3 flag spoofs the source IP address in the crafted packet?

- A. `-R`
- B. `-a`
- C. `-c`
- D. `-p`

Answer: B. The `-a` flag in hping3 sets the spoofed source IP address used in the crafted packet.

- A is wrong: `-R` sets the RST flag on the packet, which is also needed for this test, but it does not spoof the source address.
- C is wrong: `-c` sets how many packets to send; it has no effect on the source address shown.
- D is wrong: `-p` sets the destination port, not the source IP address.

Question 28. A security team wants to speed up triage of thousands of leaked session cookies found in an info-stealer malware dump, and also wants to scale up convincing phishing emails that lure victims into clicking a session-fixation link. Which statement about AI's role here is most accurate for the CEH v13 AI exam?

- A. AI eliminates the need for session ID hardening, because it can detect every hijack automatically
- B. AI can help an attacker sort stolen cookies and write convincing lure text faster, and help a defender write detection queries faster, but the underlying session weaknesses must still be fixed with proper controls
- C. AI-generated phishing emails cannot be stopped by any technical control
- D. AI is only relevant to Module 9 (Social Engineering) and has no role in session hijacking

Answer: B. The exam's AI angle treats AI as a force multiplier on both sides: it speeds up pattern-finding and lure-writing for attackers, and speeds up detection/query-writing for defenders, but does not replace the actual session hardening controls from section 3.25.

- A is wrong: AI does not remove the need for session ID hardening; weak tokens, missing timeouts, and plaintext sessions remain exploitable regardless of any AI involved.
- C is wrong: phishing-resistant MFA (such as FIDO2/WebAuthn) and URL/domain filtering can still stop AI-written phishing lures from succeeding, even if the email text itself is more convincing.
- D is wrong: this module's own AI angle section (3.32) explicitly ties AI to session hijacking lures, token pattern analysis, and detection, not only to the Social Engineering module.

Question 29. An analyst wants to confirm a suspected RST hijacking attack inside a large Wireshark capture by isolating packets that both carry a TCP reset flag and that Wireshark itself has already flagged as analysis anomalies. Which pair of display filters best achieves this?

- A. `tcp.port==23` and `ip.addr==10.0.0.5`
- B. `tcp.flags.reset==1` and `tcp.analysis.duplicate_ack`
- C. `arp.duplicate-address-detected` and `dns`
- D. `udp.port==53` and `icmp`

Answer: B. `tcp.flags.reset==1` isolates reset packets directly, and `tcp.analysis.duplicate_ack` shows where Wireshark has already flagged two different senders answering for the same sequence number, together matching the exact symptoms of RST hijacking and session desynchronization described in sections 3.3 and 3.19.

- A is wrong: these filters only narrow traffic to a port and an address; neither one looks at the RST flag or at desynchronization.
- C is wrong: this pair targets ARP spoofing and generic DNS traffic, not a TCP reset-based attack.
- D is wrong: this pair targets DNS and ICMP traffic, unrelated to the TCP RST flag this question asks about.

Question 30. An attacker collects 50 session tokens issued by a target website and pastes them into an AI assistant, asking it to find the pattern and predict the next 20 tokens. Which weakness from this module's token-prediction section is the attacker directly exploiting with this AI-assisted approach, and what is the correct server-side fix?

- A. Lack of rate limiting; the fix is to use HTTPS everywhere
- B. A predictable token-generation pattern (sequential, timestamp-based, or weak PRNG); the fix is a cryptographically secure random generator

- C. Session fixation; the fix is to regenerate the SID at login
- D. The Forbidden attack; the fix is to avoid TLS nonce reuse

Answer: B. Feeding captured tokens to an AI assistant to spot a pattern is simply a faster version of the manual pattern analysis described in section 3.7; the underlying weakness is a predictable token-generation method, and the fix is a CSPRNG with long, random tokens, not a pattern an AI (or a human) can learn from a sample.

- A is wrong: rate limiting is a separate, additional weakness (it lets an attacker test many guesses); HTTPS encrypts transport but does nothing to make the token values themselves unpredictable.
- C is wrong: session fixation involves luring a victim to use a pre-known SID, not analyzing a sample of captured tokens for a pattern.
- D is wrong: the Forbidden attack exploits TLS nonce reuse to forge HTTPS content; it has no connection to analyzing a batch of session tokens for a counting or timestamp pattern.

10. Flashcards

Q: What is session hijacking? | A: An attacker seizes control of a valid, already-authenticated TCP session between two computers. Q: What is the key difference between spoofing and hijacking? | A: Spoofing starts a new session with stolen credentials; hijacking takes over an existing, active session. Q: Name the five stages of the session hijacking process, in order. | A: Sniff, Monitor, Session Desynchronization, Session ID Prediction, Command Injection. Q: What does "session desynchronization" mean here? | A: Breaking the victim's side of the connection so the attacker's spoofed packets are accepted instead. Q: What is network-level hijacking? | A: Intercepting packets during transmission between a client and server in a TCP or UDP session. Q: What is application-level hijacking? | A: Gaining control over the HTTP user session by obtaining the session ID. Q: Name four ways a session token can become predictable. | A: Sequential pattern, timestamp-based, small token space (brute force), weak/predictable PRNG. Q: How does a MITM attack split a connection? | A: Into a client-to-attacker connection and a separate attacker-to-server connection. Q: What does Man-in-the-Browser (MitB) use to attack? | A: A Trojan horse that intercepts calls between the browser and its security mechanisms or libraries. Q: What browser feature does a MitB Trojan use to read/edit form data? | A: The DOM (Document Object Model). Q: How many steps does the module give for the Man-in-the-Browser attack? | A: 14 steps. Q: What happens in a session replay attack? | A: The attacker captures a user's authentication token and resends it later to gain unauthorized access. Q: What is session fixation? | A: Luring a victim to authenticate using a session ID the attacker already knows (fixed), then reusing that ID. Q: Name the three ways to deliver a fixed session ID. | A: Session token in the URL argument, in a hidden form field, or in a cookie. Q: What is session hijacking using proxy servers? | A: An attacker lures a victim to a bogus server that relays the whole transaction to the real server, capturing

session data. Q: What is the Forbidden attack? | A: A MITM attack on HTTPS sessions that exploits reuse of a cryptographic nonce during the TLS handshake. Q: What is a session donation attack? | A: An attacker donates their own logged-in session ID to the victim, linking the victim's actions back to the attacker's account. Q: Name the six network-level hijacking techniques. | A: Blind hijacking, UDP hijacking, TCP/IP hijacking, RST hijacking, MITM (packet sniffer), IP spoofing (source routed packets). Q: What does TCP/IP hijacking require about the attacker's location? | A: The attacker must be on the same network as the victim. Q: What is the formula for the next TCP ACK number? | A: Next ACK = previous SEQ + length of data sent. Q: What is IP spoofing using source-routed packets? | A: Spoofing a trusted host's IP and using IP source routing so replies route back through the attacker. Q: What is RST hijacking? | A: Injecting a forged, authentic-looking TCP reset packet with a spoofed source and predicted ACK to kill a connection. Q: Which two tools does the module name for RST hijacking? | A: Colasoft Packet Builder (crafting) and tcpdump (analysis). Q: What makes blind hijacking "blind"? | A: The attacker can inject data or commands but cannot see the server's response. Q: What is UDP hijacking? | A: Sending a forged server reply to a victim's UDP request before the real server's reply arrives. Q: What two techniques does the module name for the forged-gateway MITM attack? | A: Forged ICMP and ARP spoofing. Q: What does PetitPotam force a domain controller to do? | A: Initiate NTLM authentication to the attacker's server via the MS-EFSRPC API. Q: Where does PetitPotam relay the captured NTLM authentication? | A: To an Active Directory Certificate Services (AD CS) server, to generate an admin-level certificate. Q: Name the two tools the module highlights first for session hijacking. | A: Hetty (MITM HTTP proxy with logs) and Caido (real-time HTTP interception). Q: Name five more session hijacking tools listed. | A: bettercap, OWASP ZAP, WebSploit Framework, sslstrip, Burp Suite. Q: How many points are in the session hijacking countermeasure checklist? | A: 14 points. Q: Which countermeasure point says never to put the SID in a URL? | A: Avoid including the session ID in the URL or query string. Q: Name three session hijacking detection tools from the module. | A: USM Anywhere, Wireshark, plus any of Quantum IPS, SolarWinds Security Event Manager, or IBM Security Network IPS. Q: What does HSTS force browsers to do? | A: Interact with the site using HTTPS only, for a set period of time. Q: What does Token Binding protect against? | A: Session hijacking, by tying a session token to the specific TLS connection it was issued on. Q: What does DNS over HTTPS (DoH) protect, and on what port? | A: Encrypts DNS queries against snooping, sent over HTTPS on port 443. Q: What is IPsec used for? | A: Authenticating and encrypting every IP packet of a session, widely used for VPNs and remote access. Q: What is the difference between AH and ESP in IPsec? | A: AH authenticates only; ESP authenticates and encrypts. Q: What is the difference between IPsec Transport mode and Tunnel mode? | A: Transport mode keeps the original IP header (host to host); Tunnel mode wraps the whole packet in a new outer header (gateway to gateway). Q: Name the five benefits of IPsec listed in the module. | A: Network-level peer authentication, data origin authentication, data integrity, data confidentiality, replay protection.

11. Glossary

ACK (Acknowledgment number): the TCP field showing the next byte of data the receiver expects, equal to the last SEQ plus the data length. AD CS (Active Directory Certificate Services): the Windows server role that issues digital certificates; PetitPotam's NTLM relay target. AH (Authentication Header): an IPsec header that authenticates the sender and data of each IP packet, with no encryption. ARP (Address Resolution Protocol): a Data Link layer protocol that maps IP addresses to MAC addresses; spoofable with no authentication. Blind hijacking: injecting data or commands into a TCP session with no ability to see the server's response. Caido: a web security auditing toolkit for intercepting and viewing HTTP requests in real time. CRIME: Compression Ratio Info-leak Made Easy, a TLS/HTTP compression side-channel attack (ADDED CONTEXT). CSPRNG: a Cryptographically Secure Pseudo-Random Number Generator, the correct way to generate session tokens. DAI (Dynamic ARP Inspection): a switch feature that validates ARP packets against a trusted binding table. DC (Domain Controller): the Windows server that holds Active Directory and processes authentication; coerced in PetitPotam. DOI (Domain of Interpretation): the shared rules and formats that IPsec's AH, ESP, and key management agree on. DOM (Document Object Model): the browser's live, in-memory structure of a web page, read and edited by Man-in-the-Browser Trojans. DoH (DNS over HTTPS): an encrypted DNS lookup method that sends queries over HTTPS, typically on port 443. DoT (DNS over TLS): an encrypted DNS lookup method over TLS, typically on TCP port 853 (ADDED CONTEXT, verify). ESP (Encapsulating Security Payload): an IPsec header providing both authentication and encryption of each IP packet. Forbidden attack: a MITM attack on HTTPS sessions that exploits reuse of a cryptographic nonce during TLS. Hetty: an HTTP toolkit used for MITM HTTP proxy attacks with logs and advanced search. HSTS (HTTP Strict Transport Security): a web security policy that forces browsers to use HTTPS only for a site. HttpOnly: a cookie attribute that hides the cookie from browser scripts, blocking XSS-based cookie theft (ADDED CONTEXT). ICMP (Internet Control Message Protocol): an IP extension used for error and diagnostic messages; forgeable for MITM redirects. IKE (Internet Key Exchange): the protocol that negotiates keys and policy for an IPsec Security Association (ADDED CONTEXT). IPsec (Internet Protocol Security): a protocol suite that authenticates and encrypts IP packets, used for VPNs and remote access. ISN (Initial Sequence Number): the random starting sequence number each side picks during a TCP three-way handshake. MitB (Man-in-the-Browser / Manipulator-in-the-Browser): a Trojan-based attack that intercepts and edits data inside the victim's browser. MITM (Man-in-the-Middle / Manipulator-in-the-Middle): an attack that intrudes into an existing connection and intercepts the exchanged messages. MS-EFSRPC: Microsoft's Encrypting File System Remote Protocol, the API PetitPotam abuses to coerce NTLM authentication. NTLM (NT LAN Manager): an older Windows challenge-response authentication protocol, relayable to other services. Nonce: a number used once; reusing one in cryptography (as in the Forbidden attack) breaks the security guarantee. PetitPotam: an attack that coerces a domain controller to authenticate to an attacker's server, relayed to AD CS for a certificate. Proxy server

hijacking: luring a victim to a bogus server that relays their traffic to the real server while capturing session data. PRNG (Pseudo-Random Number Generator): an algorithm that produces number sequences from a seed; weak PRNGs make tokens guessable. RST hijacking: injecting a forged TCP reset packet with a spoofed source and predicted ACK to kill a victim's connection. Secure (cookie attribute): instructs the browser to send a cookie only over HTTPS, never plain HTTP (ADDED CONTEXT). SameSite (cookie attribute): limits when a cookie is sent on cross-site requests, reducing CSRF risk (ADDED CONTEXT). SEQ (Sequence number): the TCP field marking the position of the first data byte in a given packet. Session donation: an attacker donates their own valid, logged-in session ID to the victim, linking the victim back to the attacker's account. Session fixation: tricking a victim into authenticating with a session ID the attacker already knows, then reusing it. Session hijacking: seizing control of a valid, already-active TCP communication session between two computers. Session ID (SID): a server-issued value that identifies an authenticated session; also called a session token. Session replay: capturing a user's authentication token and resending it later to gain unauthorized access. Source routing: an IP option letting the sender specify the path a packet takes, usable to route replies back to a spoofing attacker. Spoofing attack: pretending to be another user or machine and starting a new session using stolen credentials. TCP/IP hijacking: using spoofed packets and a predicted sequence number to seize control of a live TCP connection on the same network. Token Binding: a mechanism that cryptographically ties a session token to the specific TLS connection it was issued on. UDP hijacking: sending a forged UDP server reply to a victim before the real server's reply arrives.

12. Coverage Report

12.1 Map of pages to guide sections

Page(s)	Topic	Guide section(s)
1	Title, learning objectives	3.1
2	What is session hijacking, hijacking process	3.2, 3.3
3	OSI model levels, spoofing vs hijacking	3.4, 3.5
4	Predicting a session token, MITM	3.7, 3.8
5	Man-in-the-Browser, session replay	3.9, 3.10
6	Session fixation, proxy server hijacking	3.11, 3.12

Page(s)	Topic	Guide section(s)
7	Forbidden attack, session donation	3.14, 3.15
8	Network-level overview, TCP/IP hijacking	3.16, 3.17
9	IP spoofing (source routed), RST hijacking	3.18, 3.19
10	Blind/UDP hijacking, MITM forged ICMP/ARP	3.20, 3.21, 3.22
11	PetitPotam, session hijacking tools	3.23, 3.24
12	14-point countermeasures, detection tools	3.25, 3.26
13	HSTS/Token Binding, DNS over HTTPS	3.27, 3.28
14	IPsec, IPsec modes and architecture	3.29, 3.30

12.2 Items marked [UNCLEAR]

- Page 9 (RST Hijacking diagram): the server's reply shows `ACK#: 1429725024`, but adding the victim's SEQ (1429775000) and LEN (24) gives 1429775024. Section 3.19 flags this as a likely slide typo or a misread digit and keeps the module's stated value while explaining the correct arithmetic.
- No other page number or hex/numeric value in the screenshots was too small or blurry to read with confidence; all diagrams, step lists, and tool names were legible.

12.3 Items to double check (gaps and verify markers)

- Slide numbers 3, 8, 9, 16 and 19 do not appear in your 14 screenshot pages (the deck jumps from slide 1 to 4, 6 to 7, 11 to 12, and 15 to 17, and 18 to 20). Sections 3.6, 3.13, 3.16, and the general intro flag this; the content for those slide numbers in this guide (session basics, the CRIME attack, and the network-level divider) is ADDED CONTEXT reconstructed from the exam blueprint and general CEH material, not from your screenshots. Watch the actual video at those points and confirm.
- Verify the PetitPotam credential-requirement note in section 3.23: the slide shows the attacker "having legitimate user's NTLM credentials" already, while some public write-ups describe unauthenticated coercion against unpatched systems. Confirm which version your course video teaches.

- Verify current IP protocol numbers for AH (51) and ESP (50), the exact IKE ports (UDP 500 and UDP 4500 for NAT traversal), and the DNS over TLS port (853), since these were added from general networking knowledge rather than shown on a slide.
- Verify current browser support and RFC status for Token Binding (RFC 8471/8472/8473), since some major browsers have removed support over time.
- Verify the exact current wording of the Microsoft KB5005413 PetitPotam mitigations (NTLM disabling, Extended Protection for Authentication) against the latest Microsoft guidance before using them in a real environment.
- Double check the tool URLs listed on the Session Hijacking Tools slide (bettercap.org, zaproxy.org, sourceforge.net for WebSploit, pypi.org for sslstrip, portswigger.net for Burp Suite) since tool hosting can move over time.

12.4 Coverage confirmation

Every screenshot page (1 to 14, covering deck slides 1-34 with repeats) has been placed into exactly one topic section in Section 3, and every named tool, command, flag, protocol, port, acronym, attack, countermeasure, and numbered step list from those pages also appears in Sections 4 through 8. No inventory item was skipped. Content for slide numbers missing from the screenshots (3, 8, 9, 16, 19) is explicitly labeled ADDED CONTEXT with a note to verify against the source video, rather than silently invented as if it came from your notes.